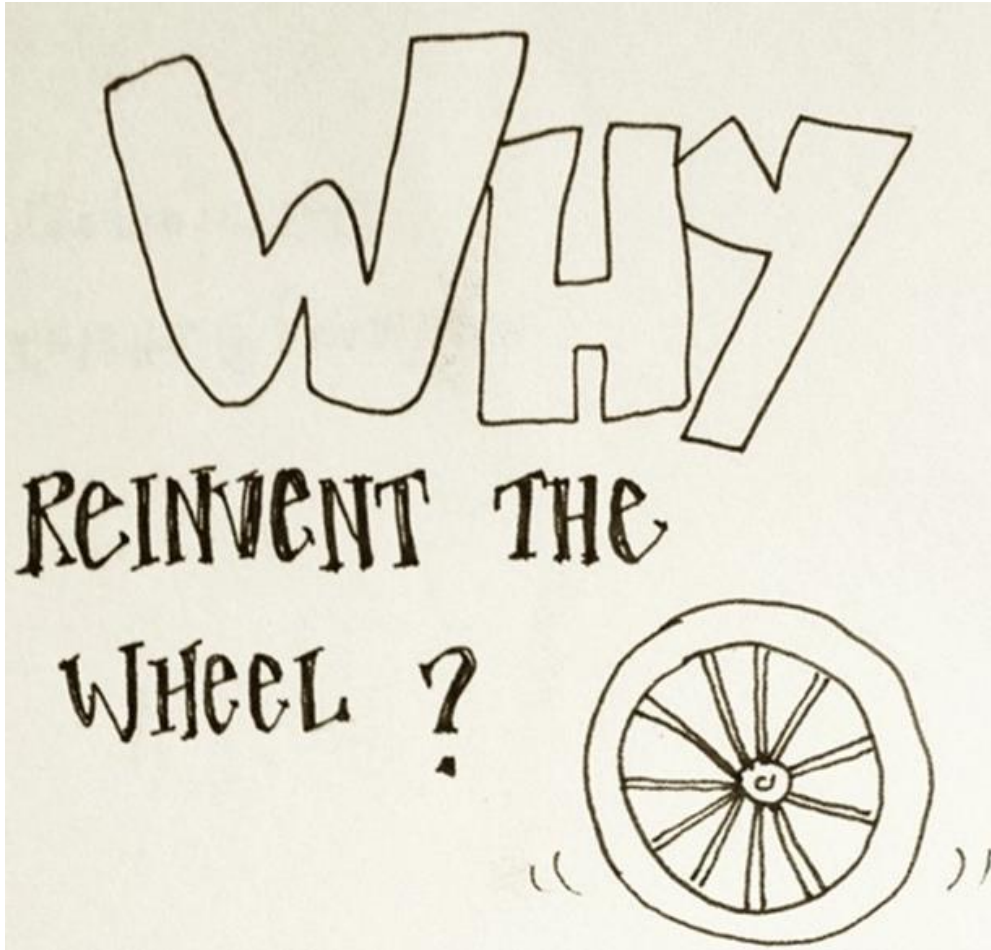


Technologies for E-Commerce

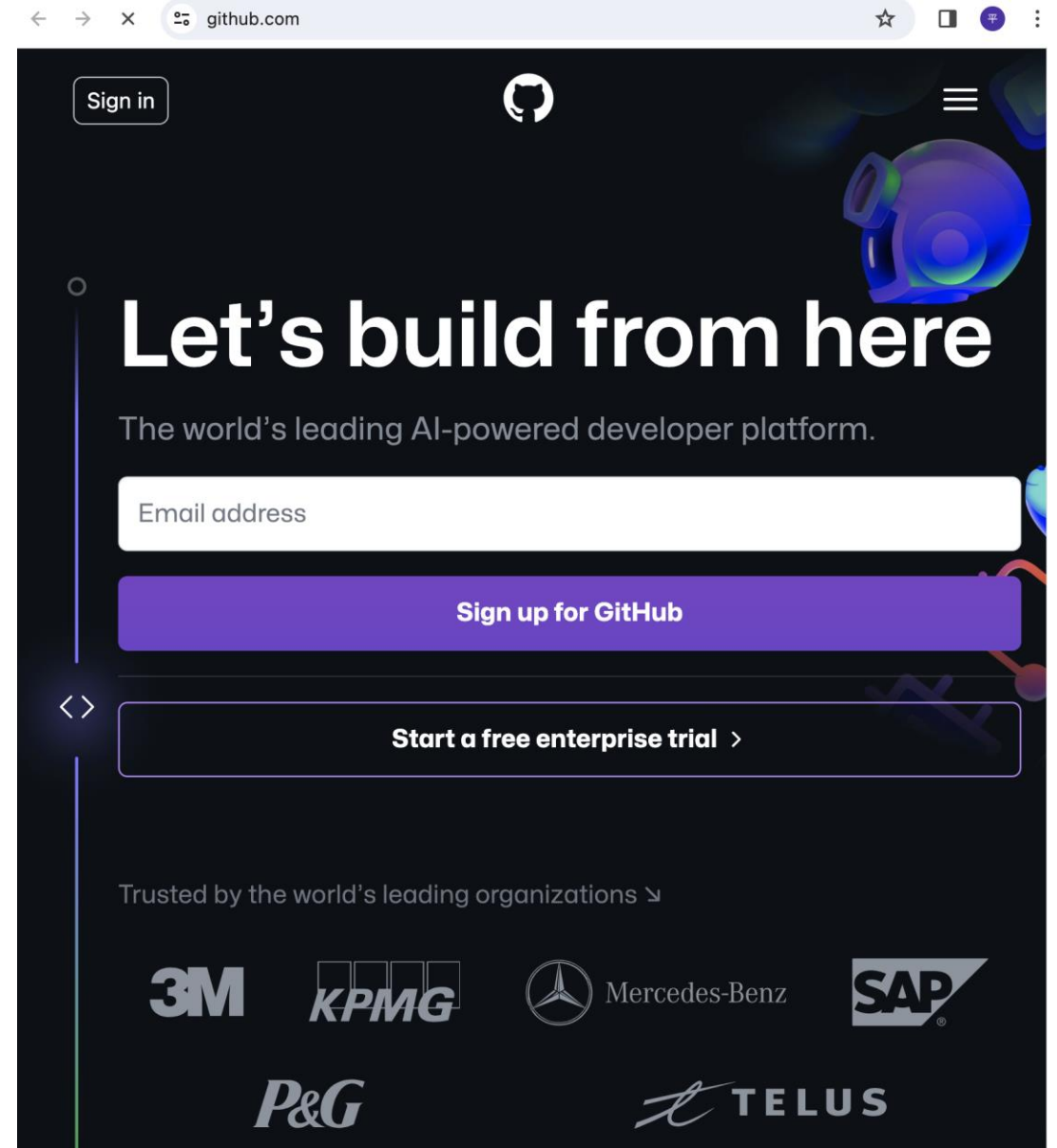
CAN302

**Department of Communications and Networking
Xi'an Jiaotong-Liverpool University (XJTLU)**

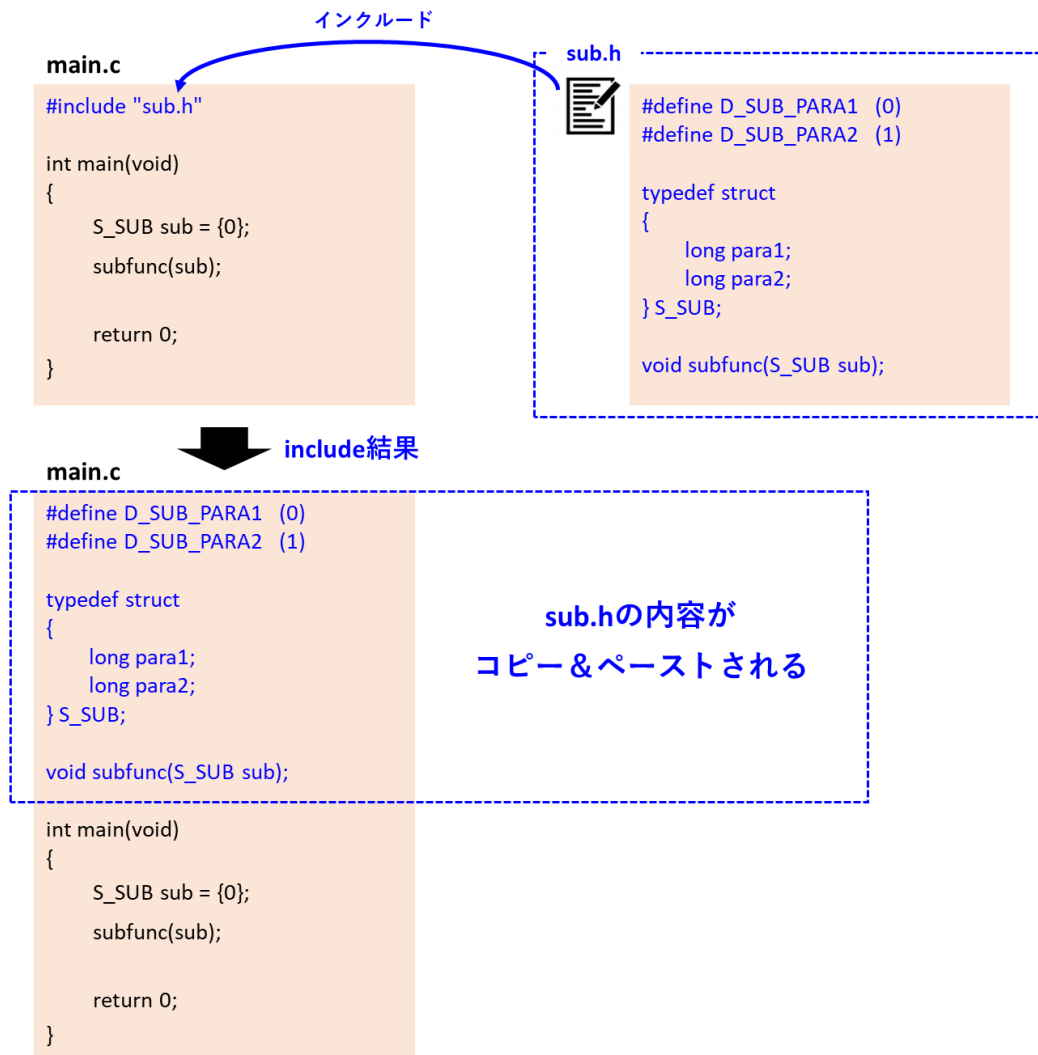
Lecture 10 – Advanced web architectures



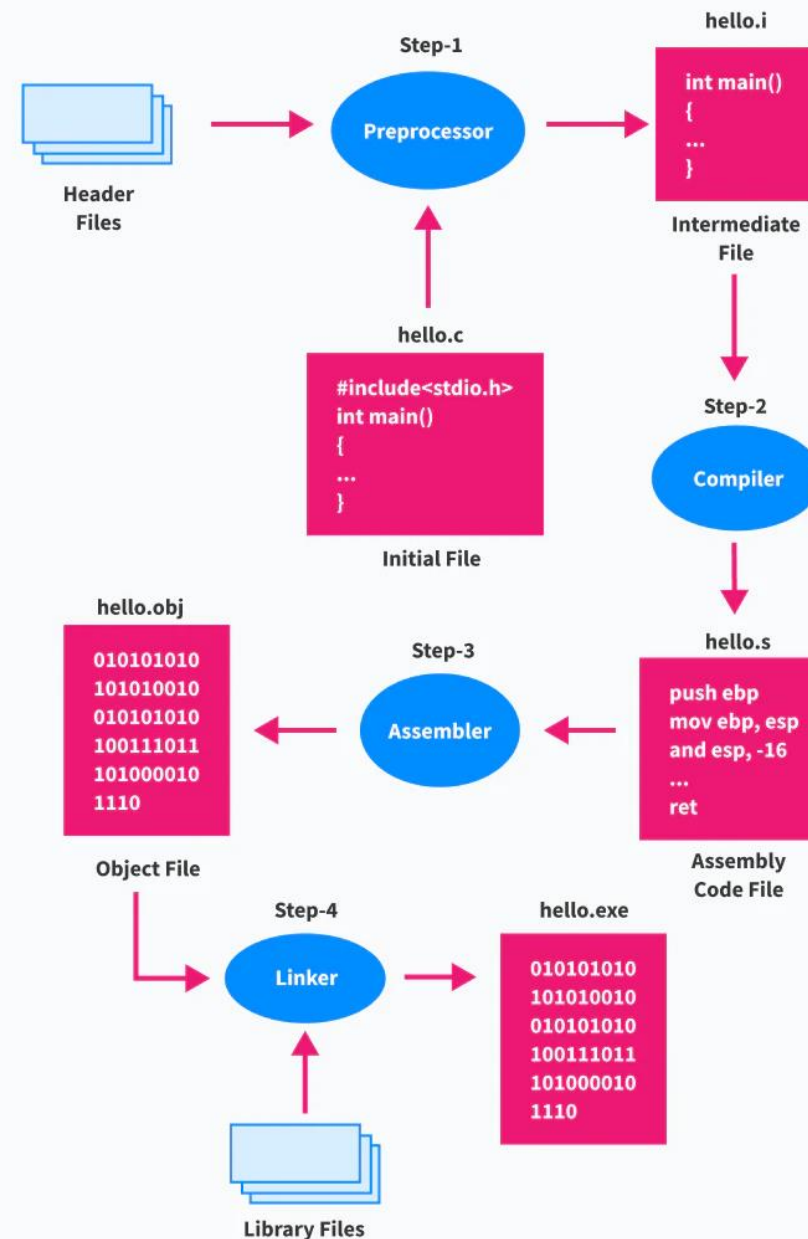
Maybe we **don't** know the wheel is **ready**!



Library in c

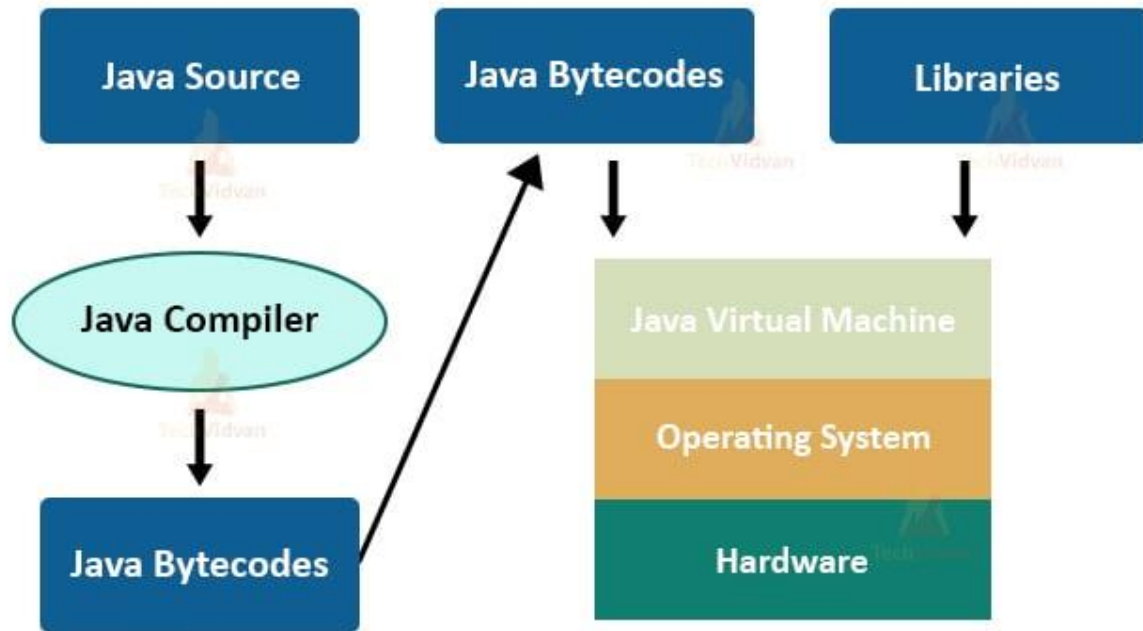


- In C language, Library is the exist “wheel”.

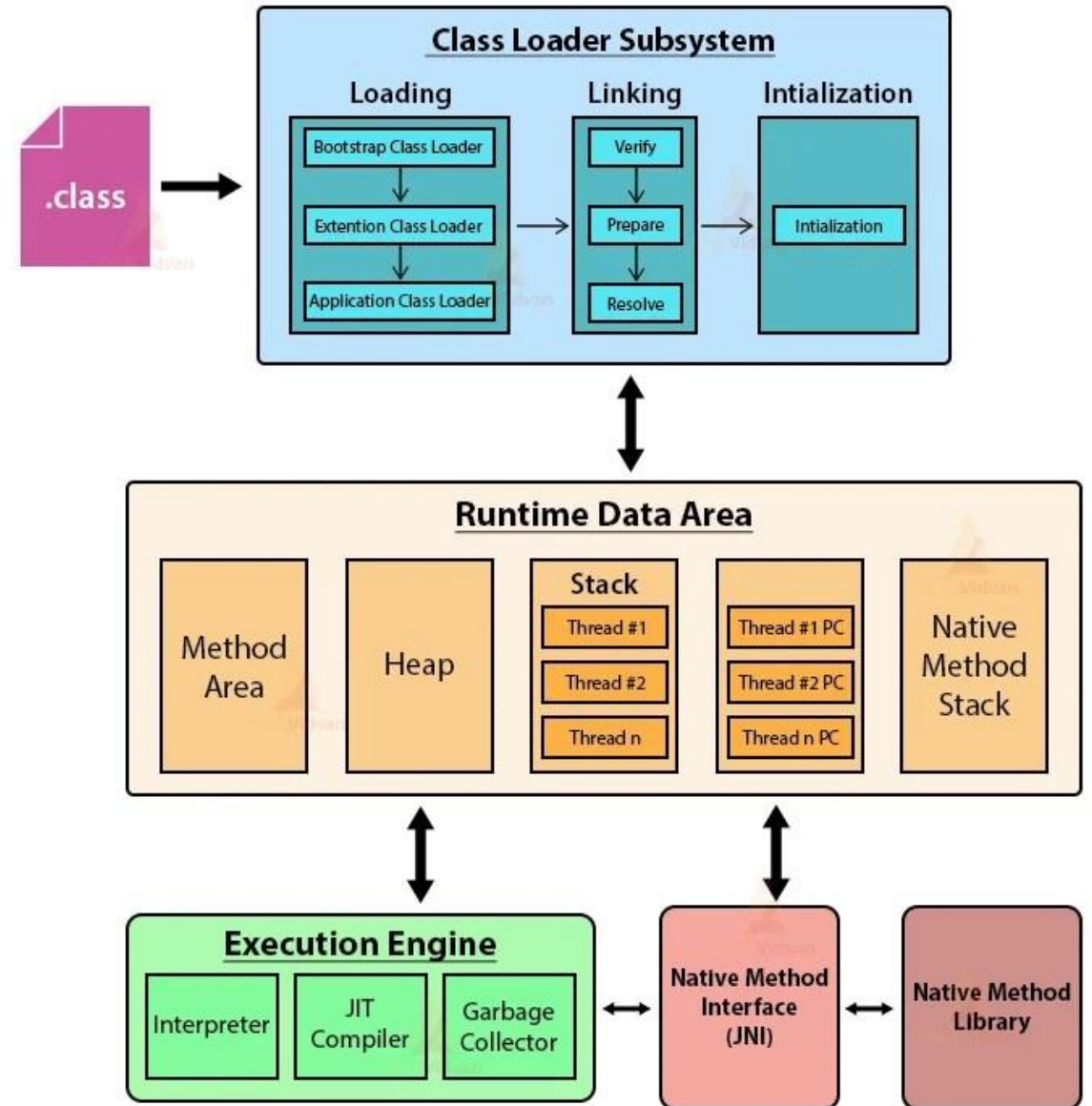


Class of Java

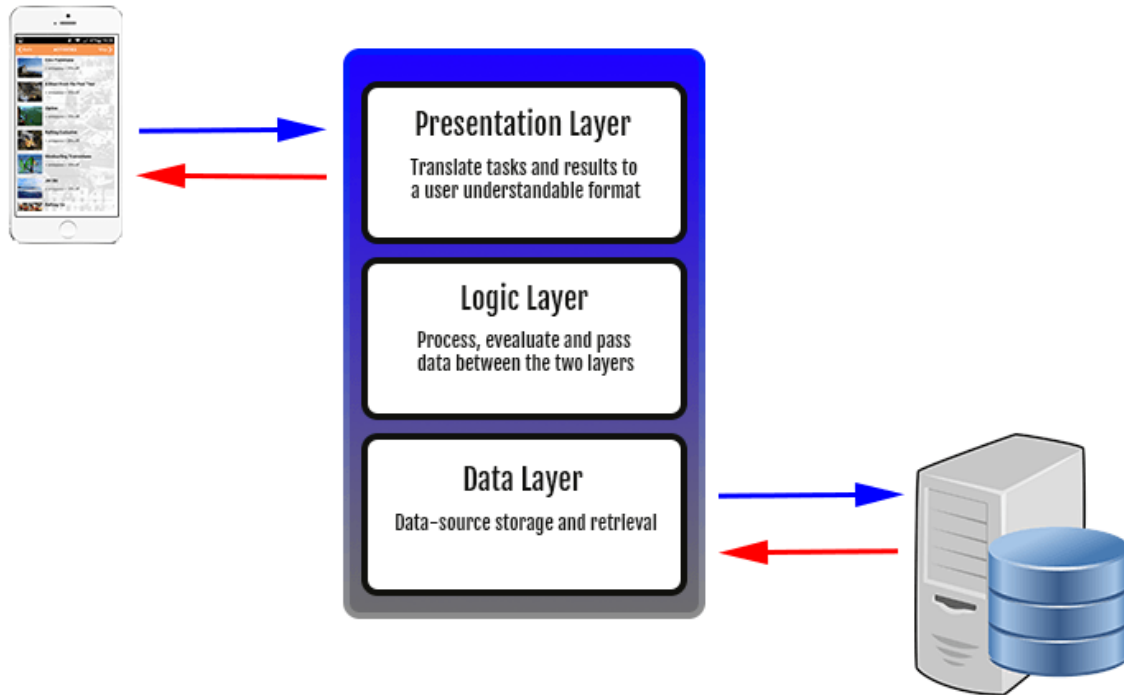
Working of JVM



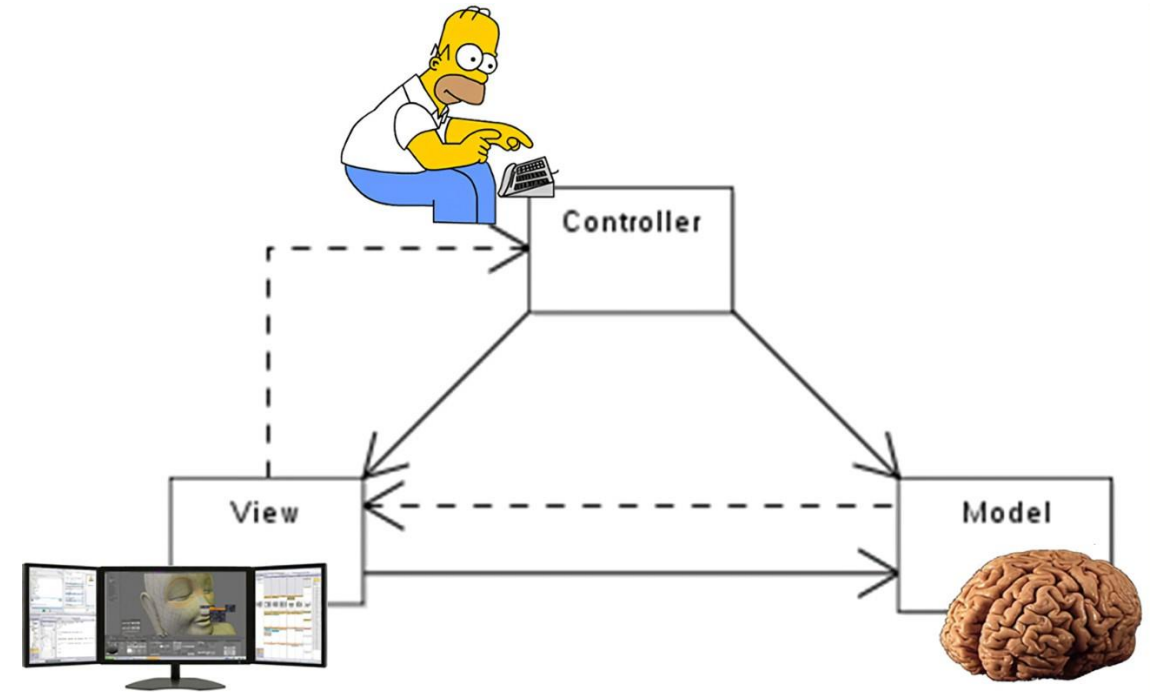
JVM Model



Split principles **within** web project



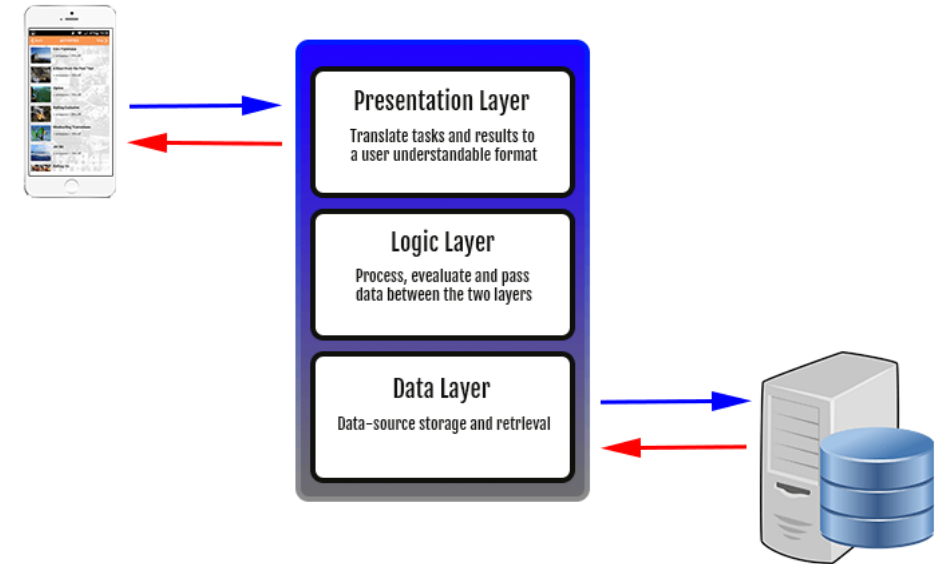
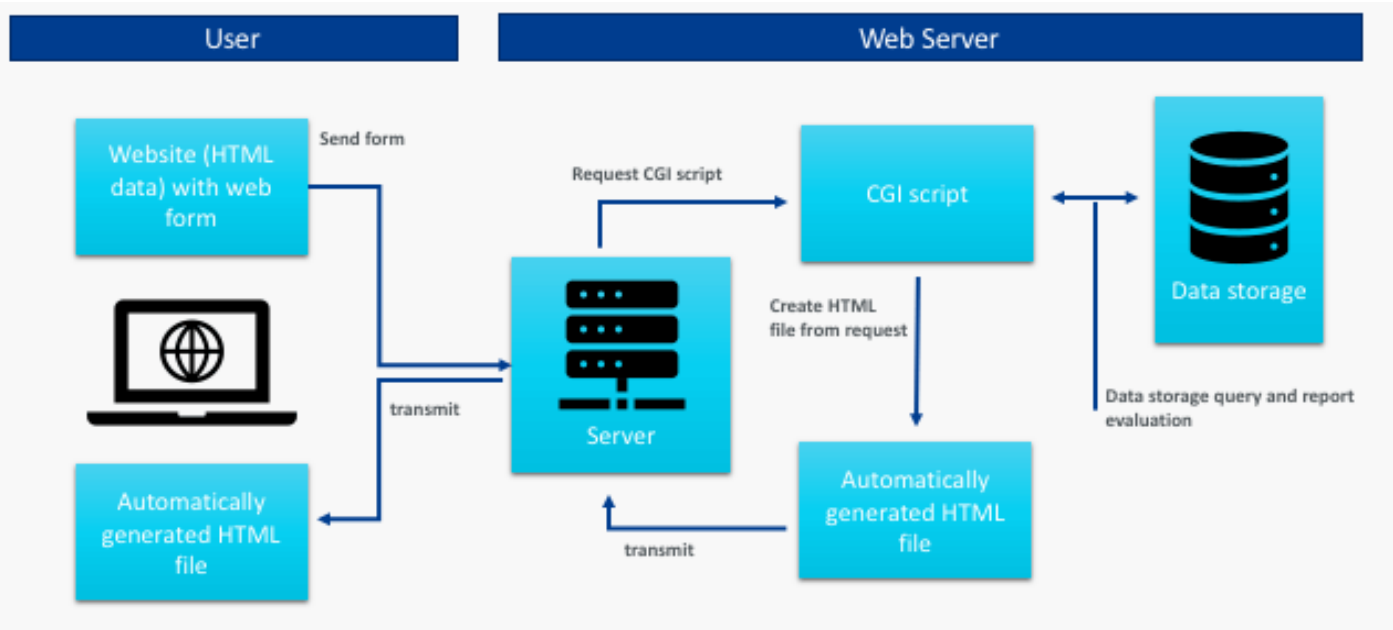
- **Three layers structure**



- **MVC structure**

Question : Does three layers corresponding to MVC?

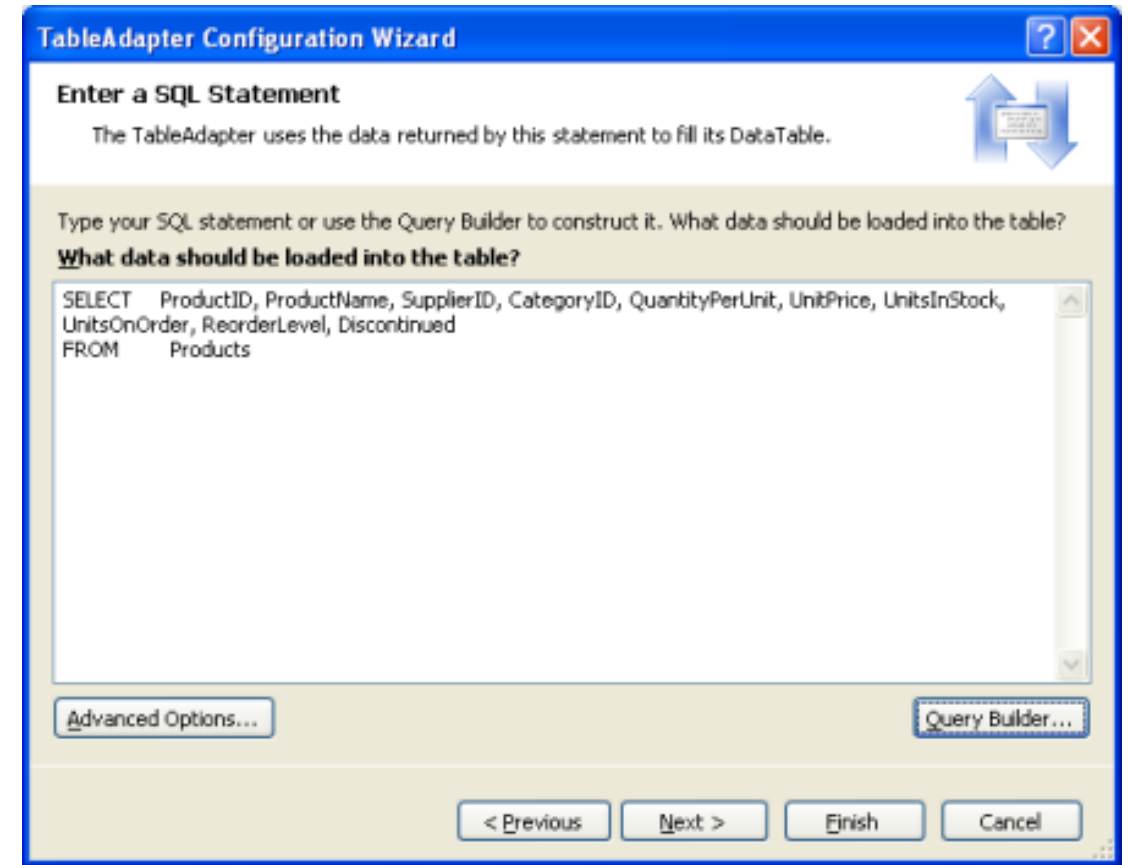
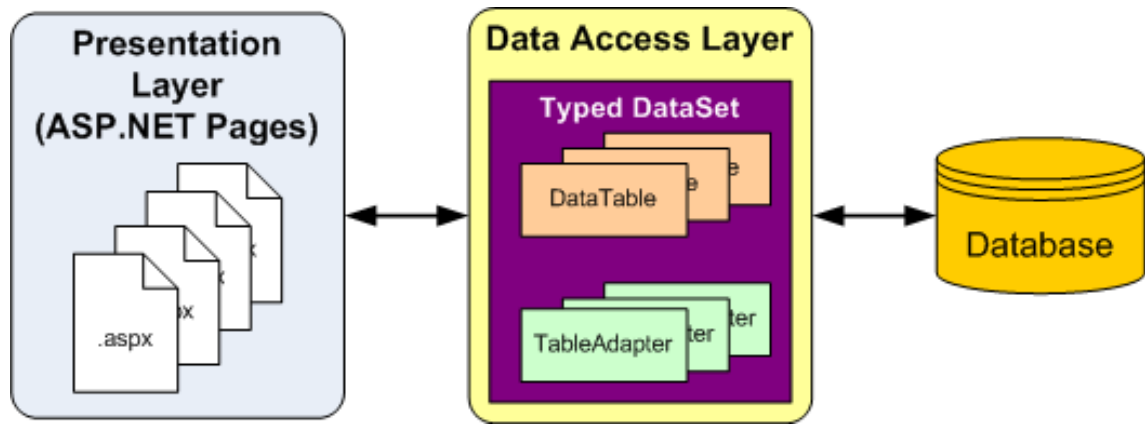
Three layers



- In our labs, the PHP file contains UI, business logic and database operation
- It can be split into different layers
- Microsoft had an architecture dividing them as: UI, BL and DA

<https://learn.microsoft.com/en-us/aspnet/web-forms/overview/data-access/introduction/>

DAL sample - 1



- Separate the data access logic from the presentation layer, which is referred to as the Data Access Layer, DAL for short, and is typically implemented as a separate Class Library project.
- All code that is specific to the underlying data source such as creating a connection to the database, issuing **SELECT**, **INSERT**, **UPDATE**, and **DELETE** commands, and so on should be located in the DAL.
- The presentation layer should not contain any references to such data access code, but should instead make calls into the DAL for any and all data requests.

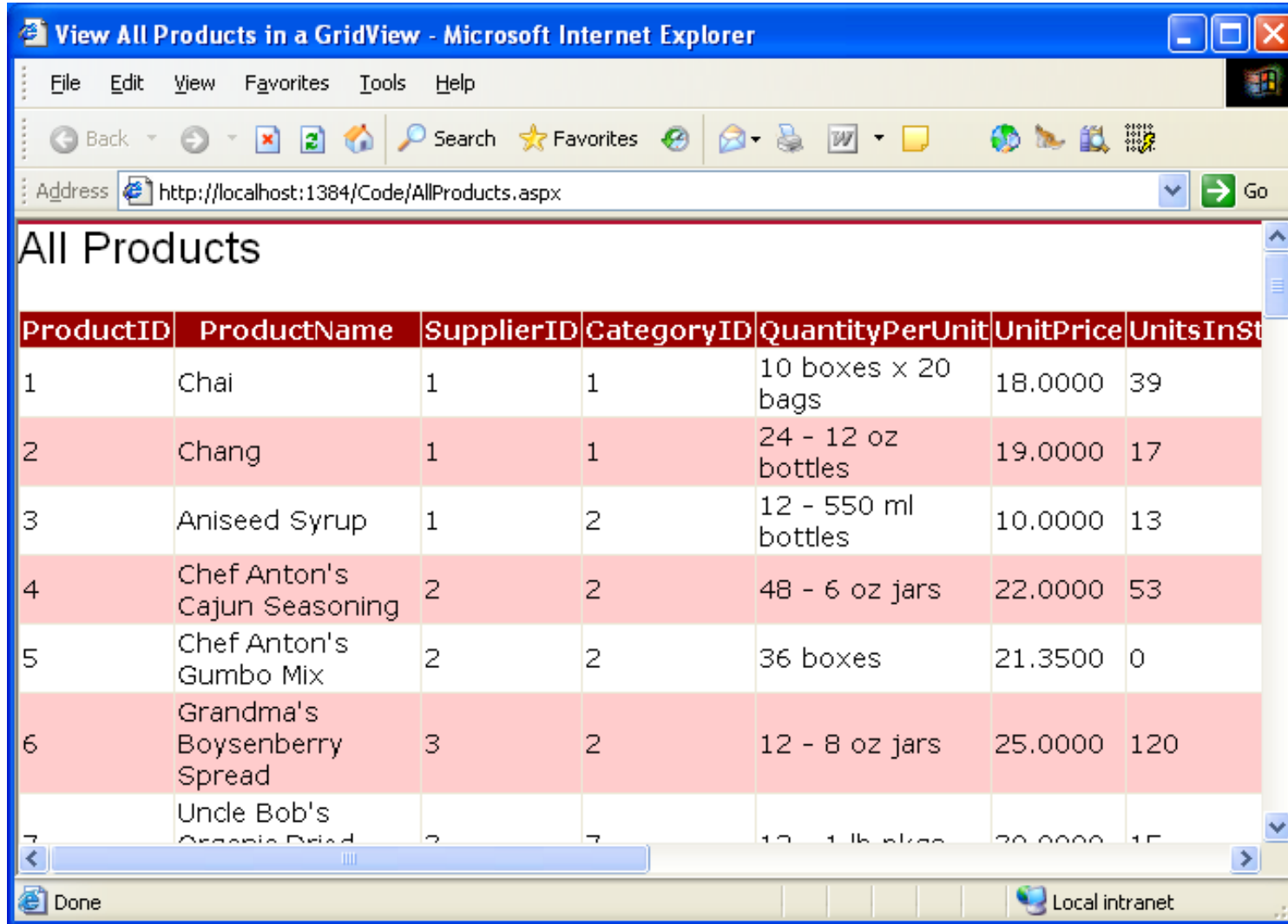
DAL sample - 2

```
test.asp
1 <%@ Page Language="C#" AutoEventWireup="true" CodeFile="AllProducts.aspx.cs"
2   Inherits="AllProducts" %>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
4   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
5 <html xmlns="http://www.w3.org/1999/xhtml" >
6 <head runat="server">
7   <title>View All Products in a GridView</title>
8   <link href="Styles.css" rel="stylesheet" type="text/css" />
9 </head>
10 <body>
11   <form id="form1" runat="server">
12     <div>
13       <h2>
14         All Products</h2>
15       <p>
16         <asp:GridView ID="GridView1" runat="server"
17           CssClass="DataWebControlStyle">
18           <HeaderStyle CssClass="HeaderStyle" />
19           <AlternatingRowStyle CssClass="AlternatingRowStyle" />
20         </asp:GridView>
21       </p>
22     </div>
23   </form>
24 </body>
25 </html>
```

```
test.cs
1 using System;
2 using System.Data;
3 using System.Configuration;
4 using System.Collections;
5 using System.Web;
6 using System.Web.Security;
7 using System.Web.UI;
8 using System.Web.UI.WebControls;
9 using System.Web.UI.WebControls.WebParts;
10 using System.Web.UI.HtmlControls;
11 using NorthwindTableAdapters;
12 public partial class AllProducts : System.Web.UI.Page
13 {
14     protected void Page_Load(object sender, EventArgs e)
15     {
16         ProductsTableAdapter productsAdapter = new
17         ProductsTableAdapter();
18         GridView1.DataSource = productsAdapter.GetProducts();
19         GridView1.DataBind();
20     }
21 }
```

- New a instant of class **ProductsTableAdapeter**.
- Then call the **GetProductus** function to get the data and bind the data to grid.
- .cs means C#, the language by MicroSoft.

DAL sample - 3



View All Products in a GridView - Microsoft Internet Explorer

Address <http://localhost:1384/Code/AllProducts.aspx> Go

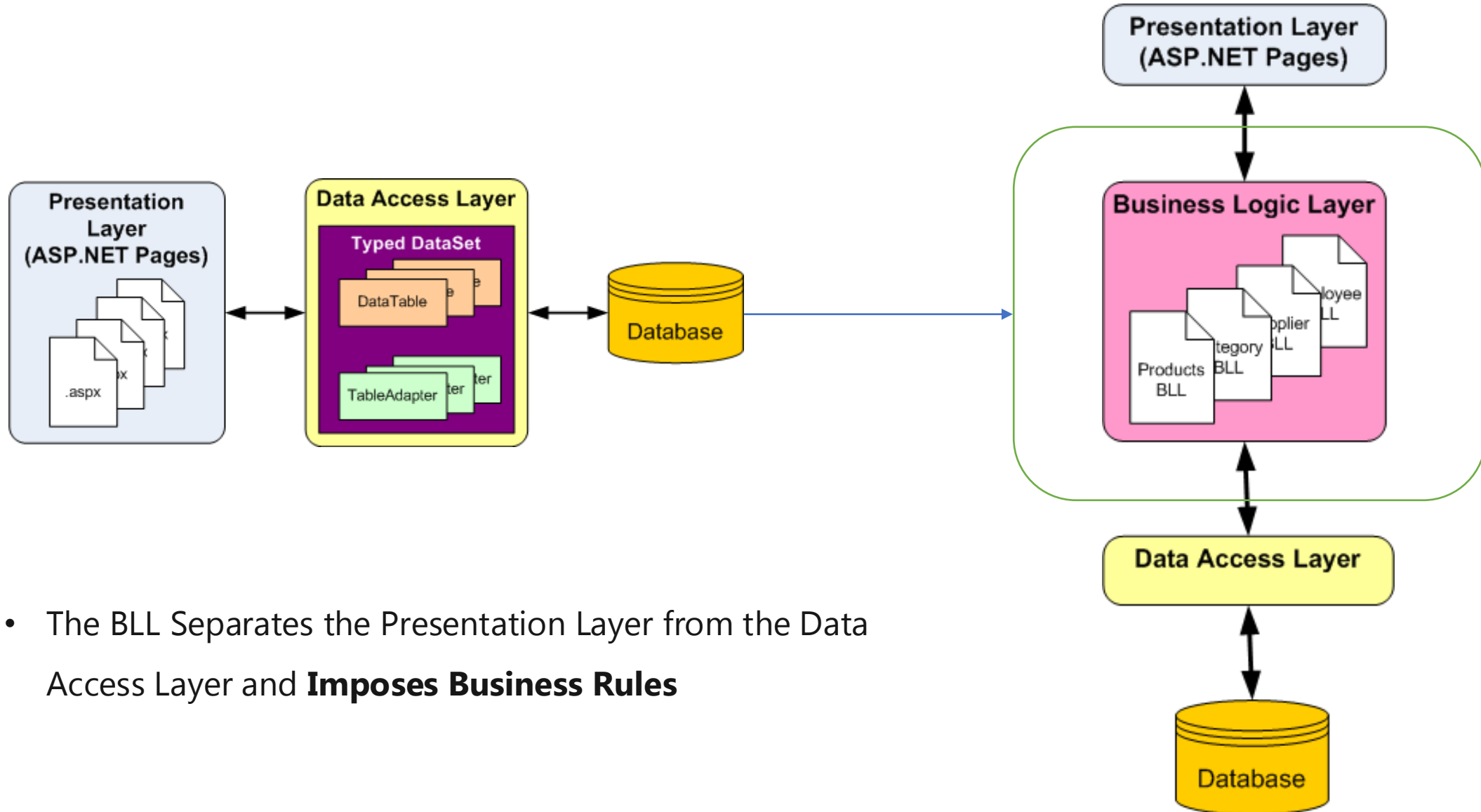
All Products

ProductID	ProductName	SupplierID	CategoryID	QuantityPerUnit	UnitPrice	UnitsInStock
1	Chai	1	1	10 boxes x 20 bags	18.0000	39
2	Chang	1	1	24 - 12 oz bottles	19.0000	17
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10.0000	13
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22.0000	53
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.3500	0
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25.0000	120
7	Uncle Bob's Organic Dried Pears	2	7	12 - 1 lb pkgs	20.0000	15

Done Local intranet

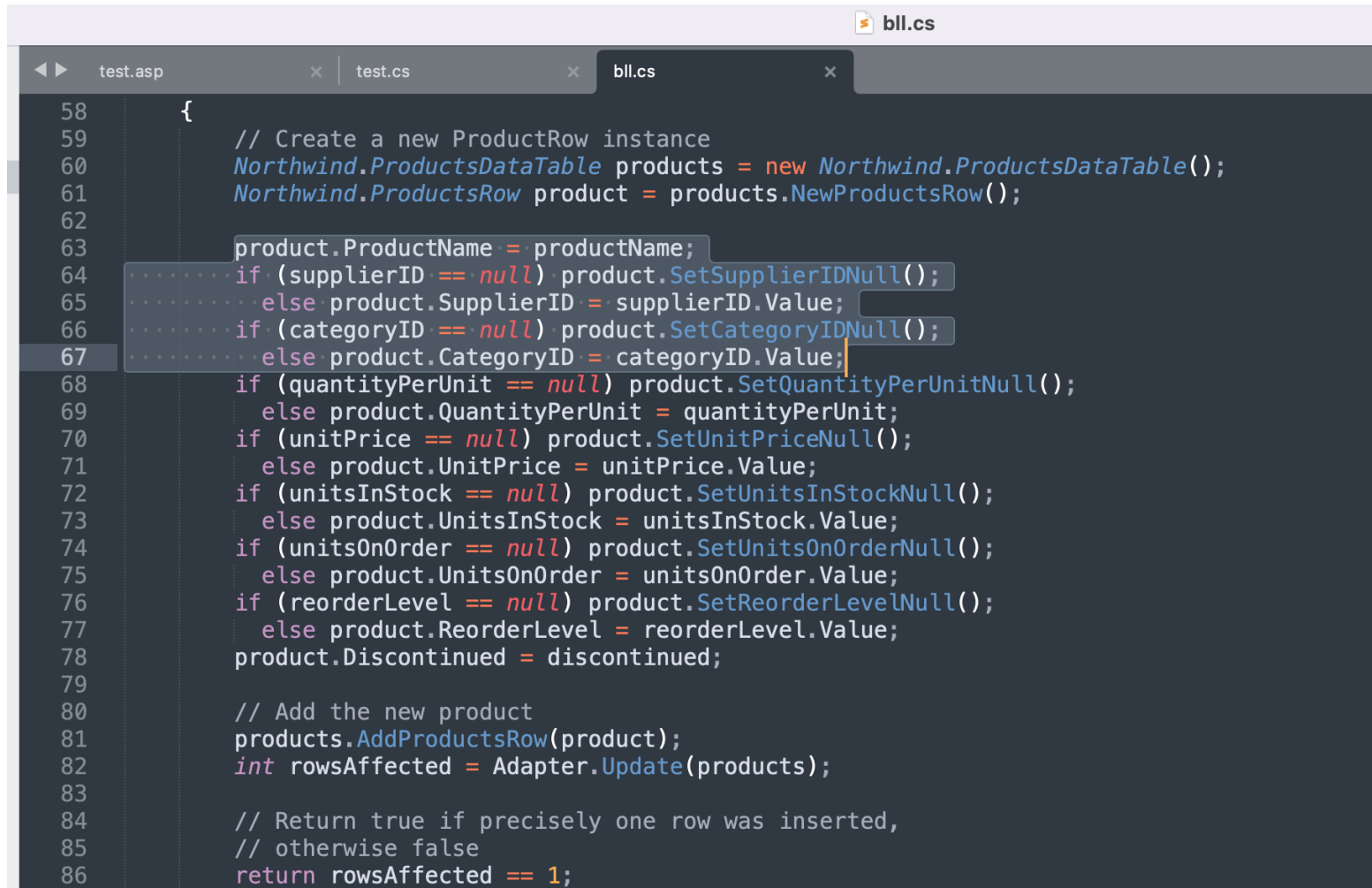
- The output by the sample code.

BLL sample - 1



- The BLL Separates the Presentation Layer from the Data Access Layer and **Imposes Business Rules**

BLL sample - 2



```
58 {
59     // Create a new ProductRow instance
60     Northwind.ProductsDataTable products = new Northwind.ProductsDataTable();
61     Northwind.ProductRow product = products.NewProductRow();
62
63     product.ProductName = productName;
64     if (supplierID == null) product.SetSupplierIDNull();
65     else product.SupplierID = supplierID.Value;
66     if (categoryID == null) product.SetCategoryIDNull();
67     else product.CategoryID = categoryID.Value;
68     if (quantityPerUnit == null) product.SetQuantityPerUnitNull();
69     else product.QuantityPerUnit = quantityPerUnit;
70     if (unitPrice == null) product.SetUnitPriceNull();
71     else product.UnitPrice = unitPrice.Value;
72     if (unitsInStock == null) product.SetUnitsInStockNull();
73     else product.UnitsInStock = unitsInStock.Value;
74     if (unitsOnOrder == null) product.SetUnitsOnOrderNull();
75     else product.UnitsOnOrder = unitsOnOrder.Value;
76     if (reorderLevel == null) product.SetReorderLevelNull();
77     else product.ReorderLevel = reorderLevel.Value;
78     product.Discontinued = discontinued;
79
80     // Add the new product
81     products.AddProductRow(product);
82     int rowsAffected = Adapter.Update(products);
83
84     // Return true if precisely one row was inserted,
85     // otherwise false
86     return rowsAffected == 1;
```

- BLL means lots of business rules (actually many if else.....)

BLL sample - 3

```
1 public partial class Northwind
2 {
3     public partial class ProductsDataTable
4     {
5         public override void BeginInit()
6         {
7             this.ColumnChanging += ValidateColumn;
8         }
9
10        void ValidateColumn(object sender,
11                             DataColumnChangeEventArgs e)
12        {
13            if(e.Column.Equals(this.UnitPriceColumn))
14            {
15                if(!Convert.IsDBNull(e.ProposedValue) &&
16                    (decimal)e.ProposedValue < 0)
17                {
18                    throw new ArgumentException(
19                        "UnitPrice cannot be less than zero", "UnitPrice");
20                }
21            }
22            else if (e.Column.Equals(this.UnitsInStockColumn) ||
23                    e.Column.Equals(this.UnitsOnOrderColumn) ||
24                    e.Column.Equals(this.ReorderLevelColumn))
25            {
26                if (!Convert.IsDBNull(e.ProposedValue) &&
27                    (short)e.ProposedValue < 0)
28                {
29                    throw new ArgumentException(string.Format(
30                        "{0} cannot be less than zero", e.Column.ColumnName),
31                        e.Column.ColumnName);
32                }
33            }
34        }
35    }
36 }
```

C#

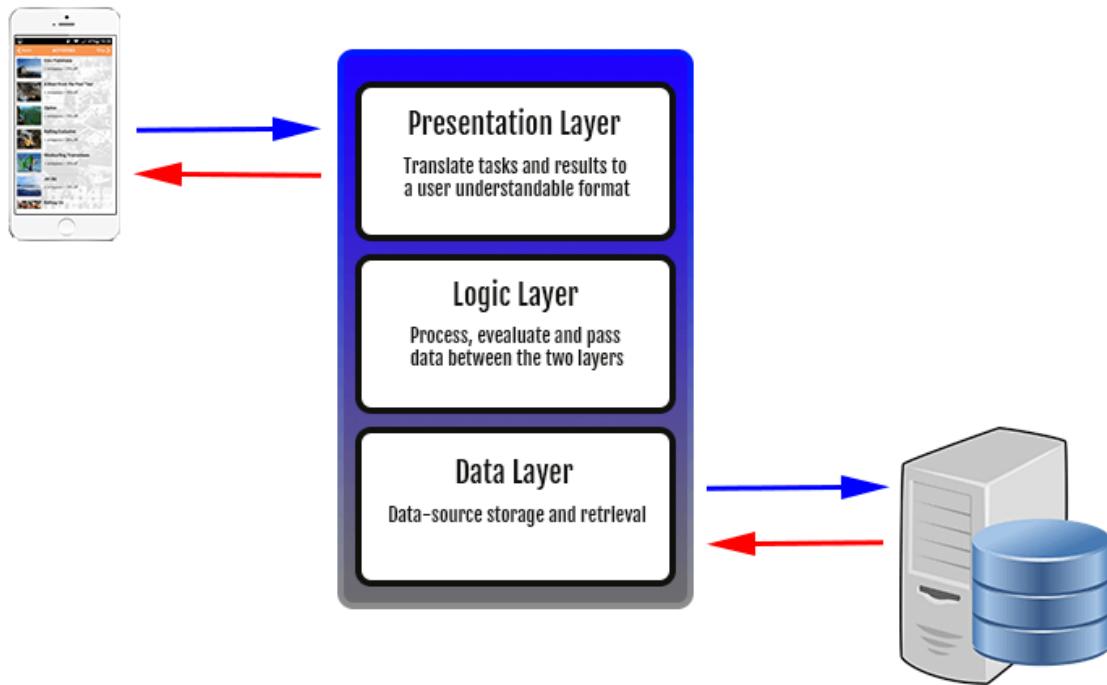
Copy

```
ProductsBLL productLogic = new ProductsBLL();

// Update information for ProductID 1
try
{
    // This will fail since we are attempting to use a
    // UnitPrice value less than 0.
    productLogic.UpdateProduct(
        "Scott s Tea", 1, 1, null, -14m, 10, null, null, false, 1);
}
catch (ArgumentException ae)
{
    Response.Write("There was a problem: " + ae.Message);
}
```

- The sample here shows: catch the error price and throw an exception

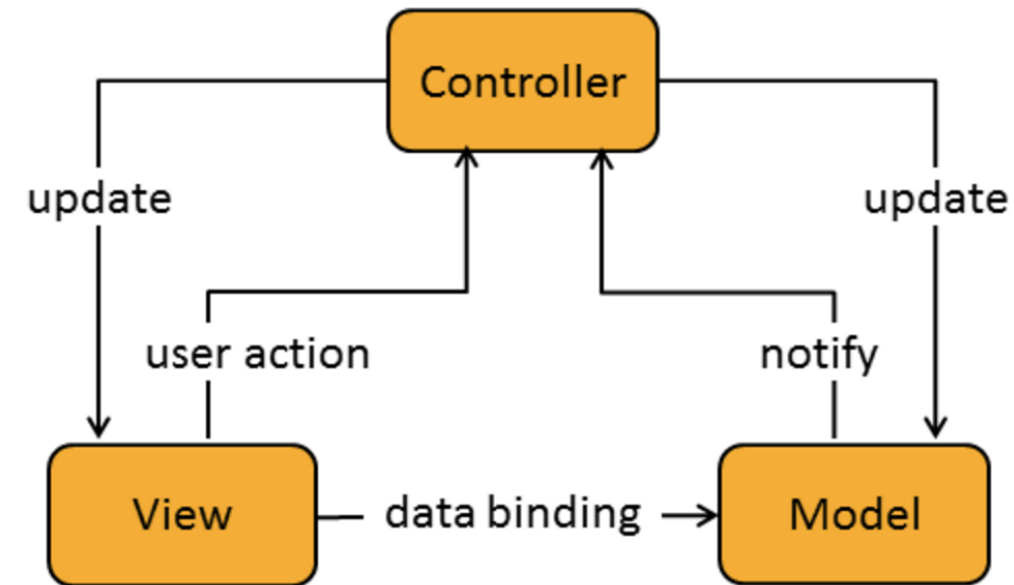
Microsoft Three layers



- It seems we need more **additional codes** to achieve the same result!
- During an application's life cycle, the three-tier approach provides benefits such as reusability, flexibility, manageability, maintainability, and scalability.
- You can divide large and complex projects into simpler projects and assign them to different programmers or programming teams.

<https://learn.microsoft.com/en-us/windows/win32/cosssdk/using-a-three-tier-architecture-model>

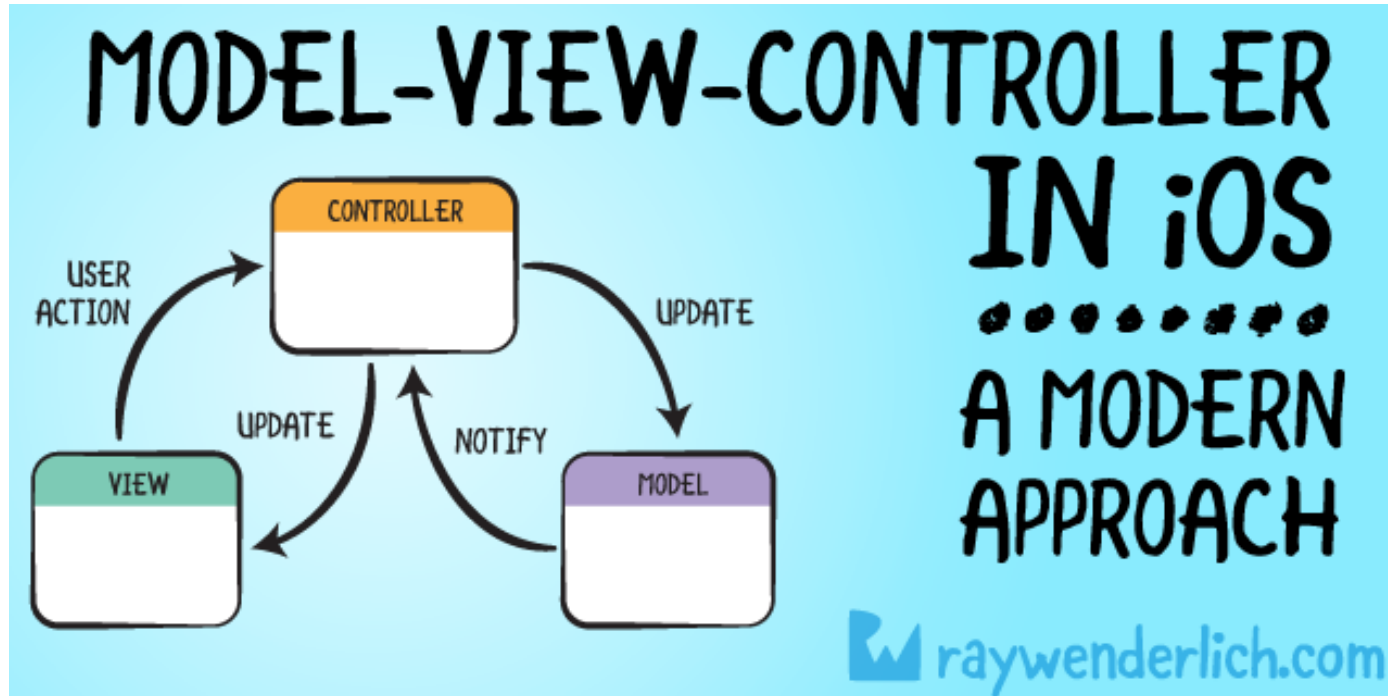
MVC



- MVC is an architecture helps to **split** applications into logical units. Actually MVC is older than web technology.
- The **model** represents a **unique entity** - it could be a single object or more likely a structure. In this way, the processing of data takes place only in the model, which ensures internal data consistency.
- The **view** is used to present the **graphical visualization** of the user interface. **To see the status of the application objects**, the view queries the model through the controller.
- The **controller** provides the **link** between the user interface (view) and the application processing logic (model). In a sense the controller enables a user to make changes and see results.
- **MVC is only a theory guiding the split.** While split means complex and cost. Follow it exactly or **NOT** depends on many factors.

<https://blogs.sap.com/2016/01/02/an-insight-into-model-view-controller-mvc-in-the-context-of-sap-ui5/>
<https://www.techopedia.com/definition/3842/model-view-controller-mvc>
<https://www.w3schools.in/mvc-architecture/>

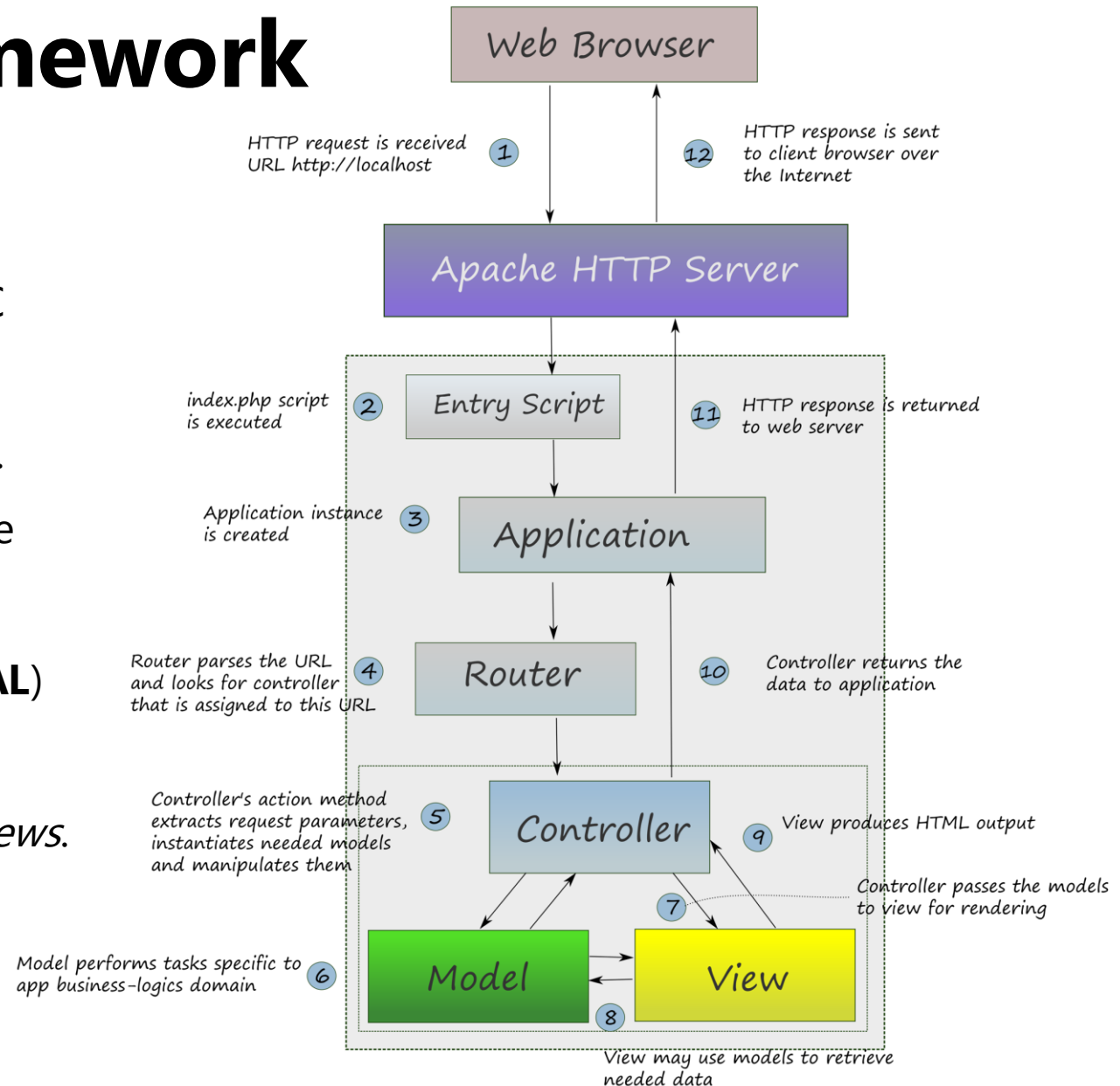
MVC is **NOT** only in WEB application



- MVC is only a theory guiding the split.
- No matter the application is a web based or client based.
- iOS app can also be developed in MVC style.
- While MVC should be in **ONE** system.

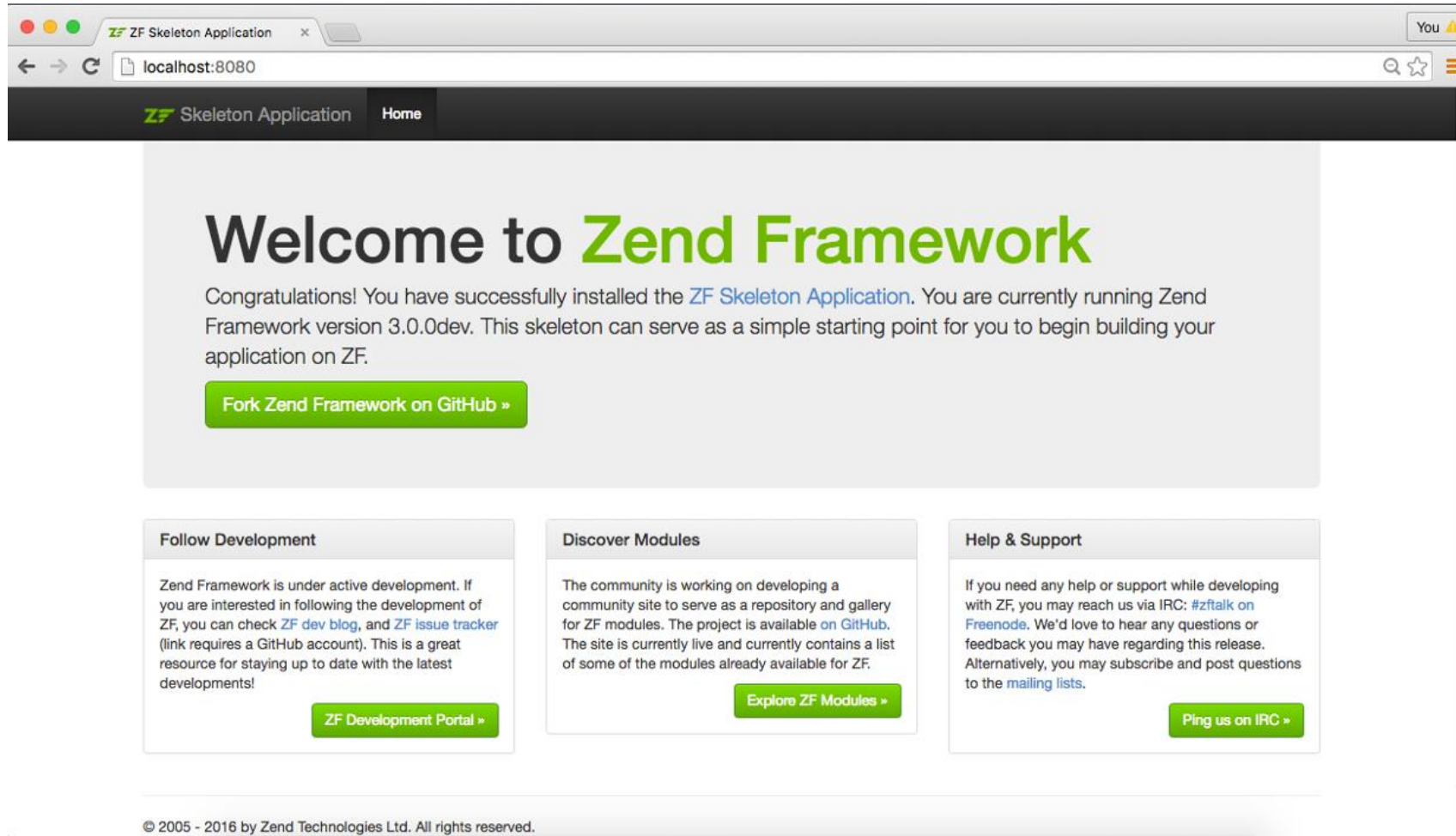
MVC sample - Zend framework

- **Zend framework** is a **framework** for PHP with MVC model support.
- It has a **router**, it replaces the default path mapping.
- The **corresponding controller** will be created by the router and corresponding method will be called.
- **Classes** implementing **business logic (including DAL)** are called *models*.
- **Code** snippets **rendering HTML** pages are called *views*.



<https://olegkrivtsov.github.io/using-zend-framework-3-book/html/en/Model View Controller/Separating Business Logic from Presentation.html>
<https://framework.zend.com/>

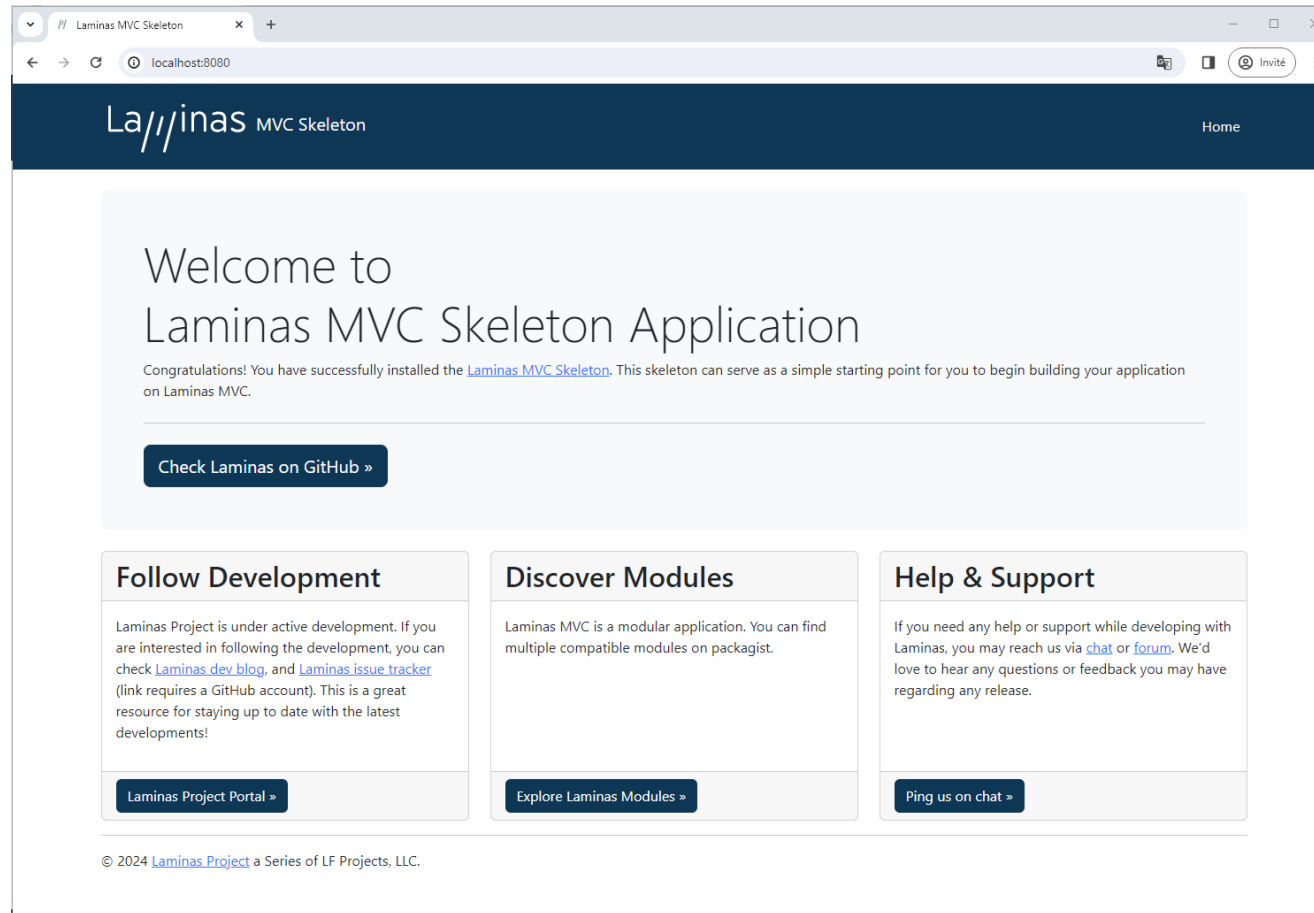
Zend framework skeleton



- Once you finish the configuration, you can see the welcome page through web browser.
- It is similar to the dashboard of XAMPP.

<https://docs.zendframework.com/tutorials/getting-started/skeleton-application/>

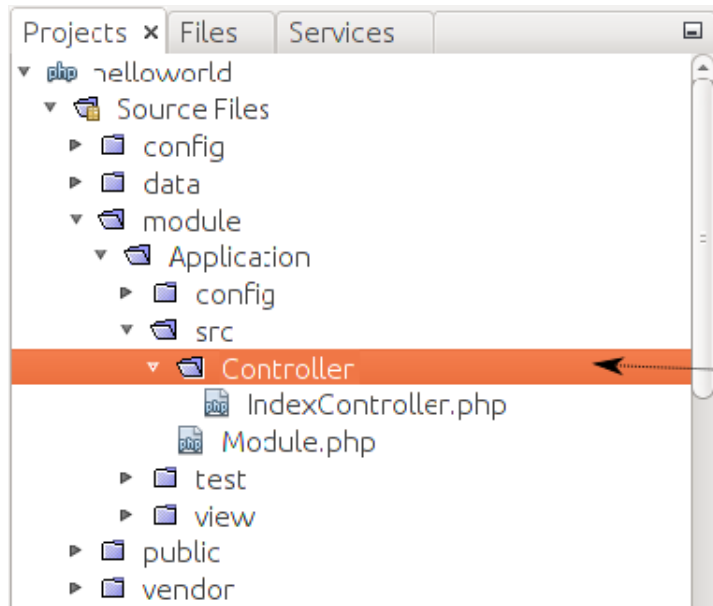
Zend framework → Laminas



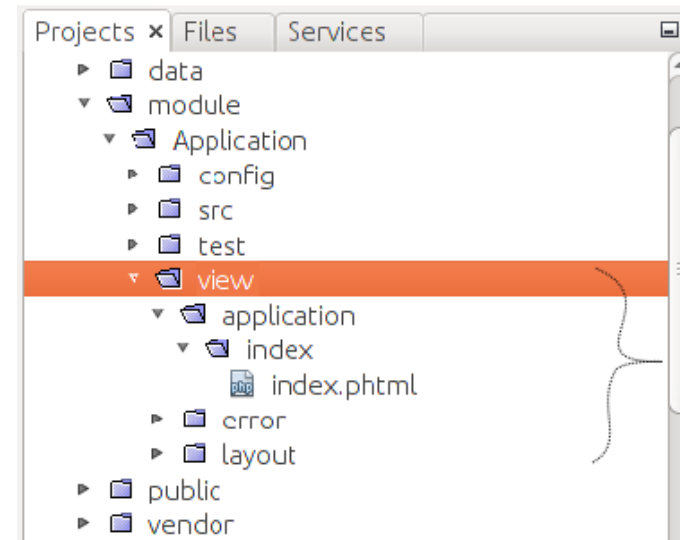
- Please follow the tutorials of latest framework Laminas

<https://docs.laminas.dev/tutorials/>

Zend framework



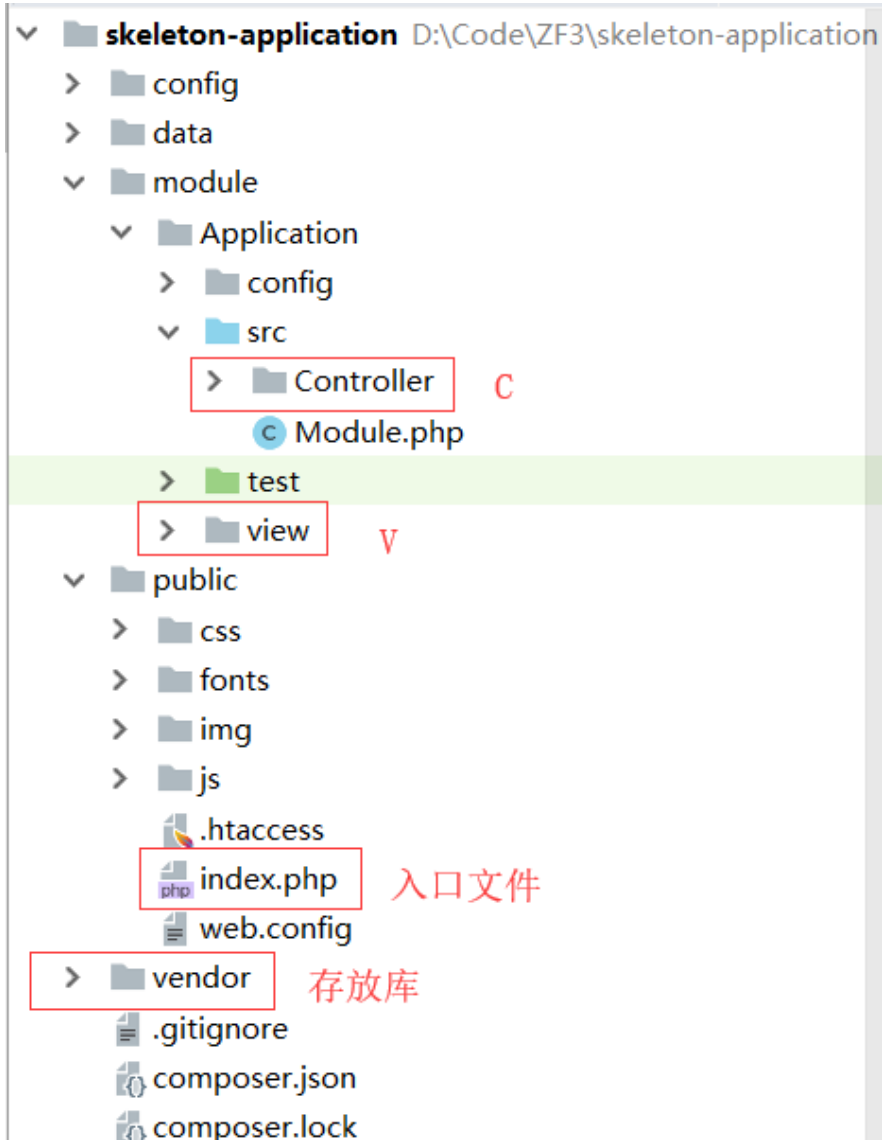
Controller classes typically live inside of "Controller" subdirectory



View templates typically live inside of module's "view" directory

- A typical file structure of a Zend framework project.
- You cannot visit `/helloworld/module/application/src/controller/IndexController.php` by the URL path.
- Url path will be used by the **router** to create the **corresponding controller** and call the corresponding method.
- The controller may interact with **Model** and return the rendered HTML code by the **View**.
- A sample will be shown how it works.

Zend framework skeleton



- Zend Framework is now the Laminas Project. We just use it for the demo purpose.
- Zend-MVC uses a module system to organize your main application-specific code within each module.
- The Application module provided by the skeleton is used to provide bootstrapping, error, and routing configuration to the whole application. It is usually used to provide application level controllers for the home page of an application.
- Public folder holds the file for public visiting.
- Vendor folder holds the libraries.

Zend framework - Module

```
1  zf-tutorial/  
2      /module  
3          /Album  
4              /config  
5              /src  
6                  /Controller  
7                  /Form  
8                  /Model  
9              /view  
10                 /album  
11                    /album
```

- This demo set up a new module – Album.
- Start by creating a directory called Album under module with the corresponding subdirectories to hold the module's files.
- Controller and Model are under src folder and view is a folder at the same level of src.
- Many configuration files need to be edited to include the Album module. Details can refer to the official documents.

Zend framework - Routing

```
1 namespace Album;
2
3 use Zend\Router\Http\Segment;
4 use Zend\ServiceManager\Factory\InvokableFactory;
5
6 return [
7     'controllers' => [
8         'factories' => [
9             Controller\AlbumController::class => InvokableFactory::class,
10        ],
11    ],
12
13    // The following section is new and should be added to your file:
14    'router' => [
15        'routes' => [
16            'album' => [
17                'type' => Segment::class,
18                'options' => [
19                    'route' => '/album[/:action[/:id]]',
20                    'constraints' => [
21                        'action' => '[a-zA-Z][a-zA-Z0-9_-]*',
22                        'id' => '[0-9]+',
23                    ],
24                    'defaults' => [
25                        'controller' => Controller\AlbumController::class,
26                        'action' => 'index',
27                    ],
28                ],
29            ],
30        ],
31    ],
32
33    'view_manager' => [
34        'template_path_stack' => [
35            'album' => __DIR__ . '/../view',
36        ],
37    ],
38];
```

- The **mapping** of a URL to a particular action is done using **routes** that are defined in the module's module.config.php file.
- The name of the route is 'album' and has a type of 'segment'.
- The segment route allows us to specify placeholders in the URL pattern (route) that will be mapped to named parameters in the matched route.
- In this case, the route is /album[/:action[/:id]] which will match any URL that starts with /album. The next segment will be an optional action name, and then finally the next segment will be mapped to an optional id. The square brackets indicate that a segment is optional.
- The constraints section allows us to ensure that the characters within a segment are as expected, so we have limited actions to starting with a letter and then subsequent characters only being alphanumeric, underscore, or hyphen. We also limit the id to digits.

Zend framework - Controller

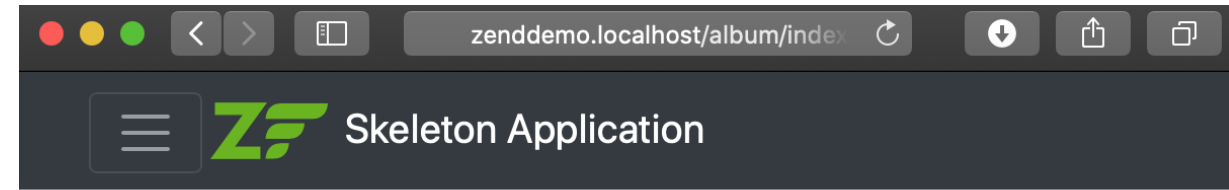
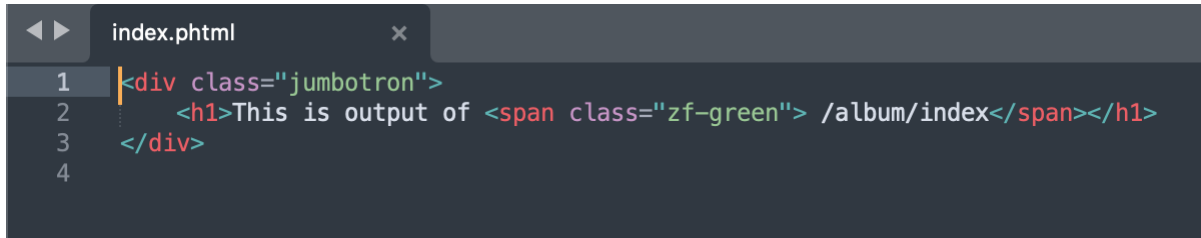
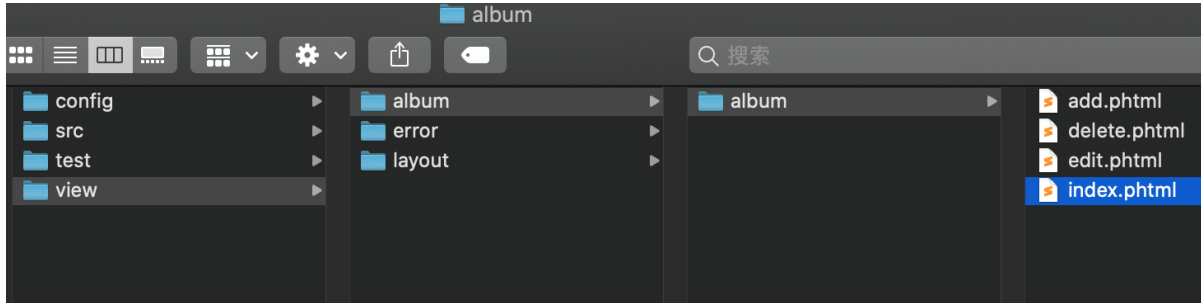
URL	Page	Action
/album	Home (list of albums)	index
/album/add	Add new album	add
/album/edit/2	Edit album with an id of 2	edit
/album/delete/4	Delete album with an id of 4	delete

```
1 namespace Album\Controller;
2
3 use Zend\Mvc\Controller\AbstractActionController;
4 use Zend\View\Model\ViewModel;
5
6 class AlbumController extends AbstractActionController
7 {
8     public function indexAction()
9     {
10    }
11
12     public function addAction()
13     {
14    }
15
16     public function editAction()
17     {
18    }
19
20     public function deleteAction()
21     {
22    }
23 }
```

- For zend-mvc, the controller is a class that is generally called {Controller name}Controller; In our case, it is AlbumController.php within the Controller subdirectory for the module.
- Each action is a public method within the controller class that is named {action name}Action, where {action name} should start with a lower case letter.
- You may see the mapping relationship for each URL with the Album controller and public methods.

<https://docs.zendframework.com/tutorials/getting-started/routing-and-controllers/>

Zend framework - View



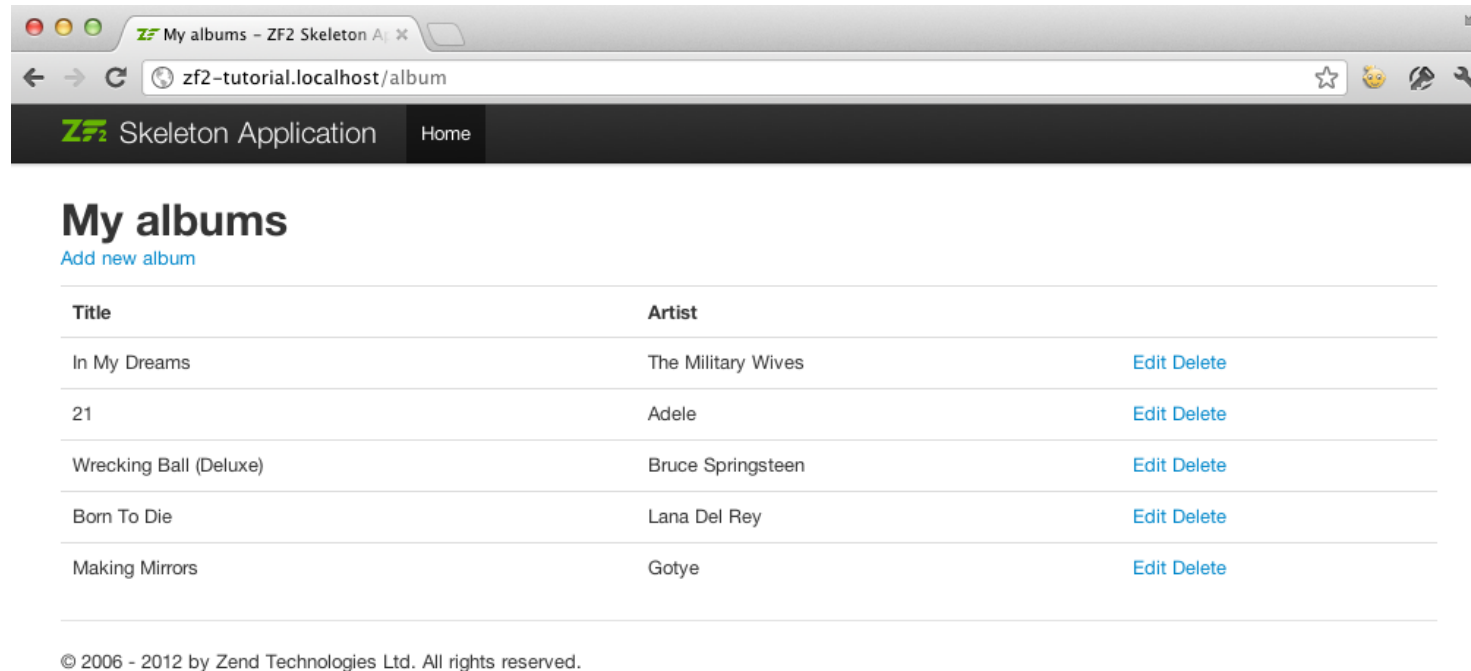
This is output of
/album/index

© 2005 - 2025 by Zend Technologies Ltd. All rights reserved.

- By all defaults, the controller will let the corresponding View render the html code.
- By defaults, the header and footer are same for all pages.

<https://docs.zendframework.com/tutorials/getting-started/routing-and-controllers/>

Zend framework - Model



Title	Artist	
In My Dreams	The Military Wives	Edit Delete
21	Adele	Edit Delete
Wrecking Ball (Deluxe)	Bruce Springsteen	Edit Delete
Born To Die	Lana Del Rey	Edit Delete
Making Mirrors	Gotye	Edit Delete

© 2006 - 2012 by Zend Technologies Ltd. All rights reserved.

- Zend Framework does not provide a zend-model component because the **model** is our business logic, and it's up to us to decide how you want it to work.
- To build the model part, we need to prepare a database with several rows at first.
- The final demo results is shown above.

<https://docs.zendframework.com/tutorials/getting-started/database-and-models/>

Zend framework – Database

db.sql

```
CREATE TABLE album (id INTEGER PRIMARY KEY AUTOINCREMENT, artist varchar(100) NOT NULL, title varchar(100) NOT NULL);
INSERT INTO album (artist, title) VALUES ('The Military Wives', 'In My Dreams');
INSERT INTO album (artist, title) VALUES ('Adele', '21');
INSERT INTO album (artist, title) VALUES ('Bruce Springsteen', 'Wrecking Ball (Deluxe)');
INSERT INTO album (artist, title) VALUES ('Lana Del Rey', 'Born To Die');
INSERT INTO album (artist, title) VALUES ('Gotye', 'Making Mirrors');
```

```
data % sqlite3 demo.db < db.sql
```

```
return [
    // ...
    'db' => [
        'driver' => 'Pdo',
        'dsn'    => sprintf('sqlite:%s/data/demo.db', realpath(getcwd())),
    ],
];
```

- We can create a sqlite3 type database with the demo data with command line tool.
- Modify config/autoload/global.php to add the db's information.

<https://docs.zendframework.com/tutorials/getting-started/database-and-models/>

Zend framework – Module

```
Album.php AlbumTable.php
<?php
namespace Album\Model;

class Album
{
    public $id;
    public $artist;
    public $title;

    public function exchangeArray(array $data)
    {
        $this->id      = !empty($data['id']) ? $data['id'] : null;
        $this->artist   = !empty($data['artist']) ? $data['artist'] : null;
        $this->title    = !empty($data['title']) ? $data['title'] : null;
    }
}
```

- We treat each Album item as an object can then have the class Album.
- Class AlbumTable is the mapping with Database through **TableGatewayInterface**.

```
Album.php AlbumTable.php
1 <?php
2 namespace Album\Model;
3
4 use RuntimeException;
5 use Zend\Db\TableGateway\TableGatewayInterface;
6
7 class AlbumTable
8 {
9     private $tableGateway;
10
11     public function __construct(TableGatewayInterface $tableGateway)
12     {
13         $this->tableGateway = $tableGateway;
14     }
15
16     public function fetchAll()
17     {
18         return $this->tableGateway->select();
19     }
20
21     public function getAlbum($id)
22     {
23         ...
24     }
25
26     public function saveAlbum(Album $album)
27     {
28         ...
29     }
30
31     public function deleteAlbum($id)
32     {
33         ...
34     }
35 }
```

Zend framework – Table gateway

```
1 namespace Zend\Db\TableGateway;
2
3 use Zend\Db\ResultSet\ResultSetInterface;
4 use Zend\Db\Sql\Where;
5
6 interface TableGatewayInterface
7 {
8     public function getTable() : string;
9     public function select(Where|callable|string|array $where = null) : ResultSetInterface;
10    public function insert(array $set) : int;
11    public function update(
12        array $set,
13        Where|callable|string|array $where = null,
14        array $joins = null
15    ) : int;
16    public function delete(Where|callable|string|array $where) : int;
17 }
```

- The Table Gateway subcomponent provides an object-oriented representation of a database table; its methods mirror the most common table operations.
- Table Gateway will use the DB information we have prepared in global configuration.

<https://docs.zendframework.com/zend-db/table-gateway/>

Zend framework – modify controller

```
1 namespace Album\Controller;
2
3 // Add the following import:
4 use Album\Model\AlbumTable;
5 use Zend\Mvc\Controller\AbstractActionController;
6 use Zend\View\Model\ViewModel;
7
8 class AlbumController extends AbstractActionController
9 {
10     // Add this property:
11     private $table;
12
13     // Add this constructor:
14     public function __construct(AlbumTable $table)
15     {
16         $this->table = $table;
17     }
18
19     /* ... */
20 }
```

- Then we can use the model in controller.
- You may notice in we did not **new** a model instance.
- If you use magic word **new**, then later on the switch to another language will be a tough job.
- A factory is abstract for new all objects instead.

Zend framework – factory

```
Module.php
4 use Zend\Db\Adapter\AdapterInterface;
5 use Zend\Db\ResultSet\ResultSet;
6 use Zend\Db\TableGateway\TableGateway;
7 use Zend\ModuleManager\Feature\ConfigProviderInterface;
8
9 class Module implements ConfigProviderInterface
10 {
11     public function getConfig()
12     { ... }
13
14
15
16     public function getServiceConfig()
17     {
18         return [
19             'factories' => [
20                 Model\AlbumTable::class => function($container) {
21                     $tableGateway = $container->get(Model\AlbumTableGateway::class);
22                     return new Model\AlbumTable($tableGateway);
23                 },
24                 Model\AlbumTableGateway::class => function ($container) {
25                     $dbAdapter = $container->get(AdapterInterface::class);
26                     $resultSetPrototype = new ResultSet();
27                     $resultSetPrototype->setArrayObjectPrototype(new Model\Album());
28                     return new TableGateway('album', $dbAdapter, null, $resultSetPrototype);
29                 },
30             ],
31         ];
32     }
33
34     public function getControllerConfig()
35     {
36         return [
37             'factories' => [
38                 Controller\AlbumController::class => function($container) {
39                     return new Controller\AlbumController(
40                         $container->get(Model\AlbumTable::class)
41                     );
42                 },
43             ],
44         ];
45     }
46 }
```

- We should register the class in the factory, so the factory can new the instance.
- Controller class registers as Controller config.
- Model class should register as Service.
- This part is very trick in MVC and this is only a simple sample. Factory can many different modes, it beyond our model.

Zend framework – modify method

```
1 // module/Album/src/Controller/AlbumController.php:
2 // ...
3     public function indexAction()
4     {
5         return new ViewModel([
6             'albums' => $this->table->fetchAll(),
7         ]);
8     }
9 // ...
```

- Zend Framework uses the ViewModel to pass data from the Controller to the View. A ViewModel is created and populated within the Controller's action method, then returned.
- This sample will pass all album data in database to view for rendering.

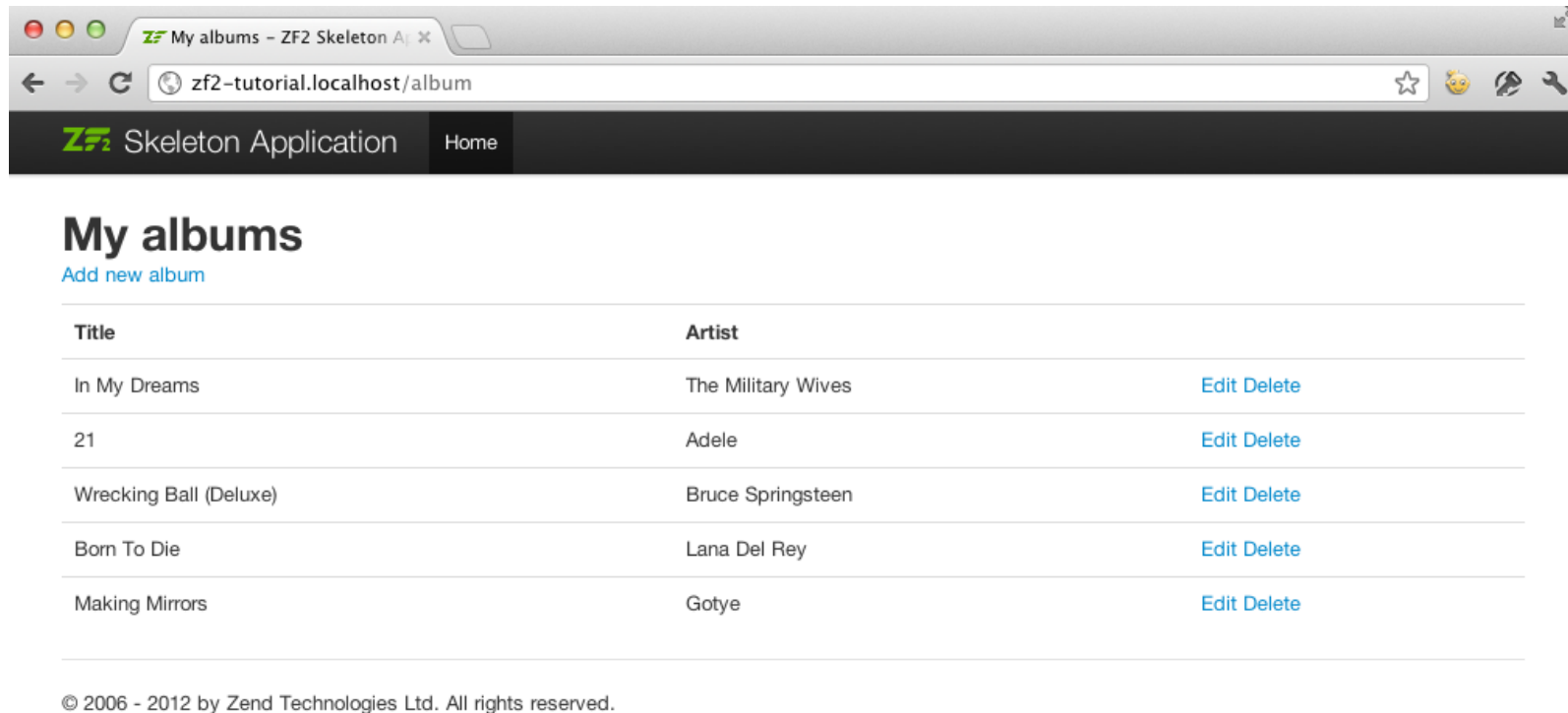
<https://docs.zendframework.com/tutorials/getting-started/database-and-models/>

Zend framework – modify the view

```
1 <?php
2 // module/Album/view/album/album/index.phtml:
3
4 $title = 'My albums';
5 $this->headTitle($title);
6 ?>
7 <h1><?= $this->escapeHtml($title) ?></h1>
8 <p>
9     <a href="<?= $this->url('album', ['action' => 'add']) ?>">Add new album</a>
10 </p>
11
12 <table class="table">
13 <tr>
14     <th>Title</th>
15     <th>Artist</th>
16     <th>&nbsp;</th>
17 </tr>
18 <?php foreach ($albums as $album) : ?>
19     <tr>
20         <td><?= $this->escapeHtml($album->title) ?></td>
21         <td><?= $this->escapeHtml($album->artist) ?></td>
22         <td>
23             <a href="<?= $this->url('album', ['action' => 'edit', 'id' => $album->id]) ?>">Edit</a>
24             <a href="<?= $this->url('album', ['action' => 'delete', 'id' => $album->id]) ?>">Delete</a>
25         </td>
26     </tr>
27 <?php endforeach; ?>
28 </table>
```

- The first thing we do is to set the title for the page (used in the layout) and also set the title for the <head> section using the headTitle() view helper which will display in the browser's title bar. We then create a link to add a new album.
- A table to show all album are output by foreach.
- We always use the escapeHtml() view helper to help protect ourselves from Cross Site Scripting (XSS) vulnerabilities.

Zend framework - result



- Then you can see the result as above.
- You should continue the rest part to make full function of this demo (add, edit and delete).

<https://docs.zendframework.com/tutorials/getting-started/database-and-models/>

Zend framework - Review

```
index.php
1 <?php
2
3 use Zend\Mvc\Application;
4 use Zend\Stdlib\ArrayUtils;
5
6 /**
7  * This makes our life easier when dealing with paths. Everything is relative
8  * to the application root now.
9  */
10 chdir(dirname(__DIR__));
11
12 // Decline static file requests back to the PHP built-in webserver
13 if (php_sapi_name() === 'cli-server') {
14     $path = realpath(__DIR__ . parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH));
15     if (($_FILE !== $path && is_file($path))) {
16         return false;
17     }
18     unset($path);
19 }
20
21 // Composer autoloading
22 include __DIR__ . '/../vendor/autoload.php';
23
24 if (!class_exists(Application::class)) {
25     throw new RuntimeException(
26         "Unable to load application.\n"
27         . "- Type `composer install` if you are developing locally.\n"
28         . "- Type `vagrant ssh -c 'composer install'` if you are using Vagrant.\n"
29         . "- Type `docker-compose run zf composer install` if you are using Docker.\n"
30     );
31 }
32
33 // Retrieve configuration
34 $appConfig = require __DIR__ . '/../config/application.config.php';
35 if (file_exists(__DIR__ . '/../config/development.config.php')) {
36     $appConfig = ArrayUtils::merge($appConfig, require __DIR__ . '/../config/development
37 }
38
39 // Run the application!
40 Application::init($appConfig)->run();
41
```

- Actually, whatever the URL you input, Zend framework will always run the /public/index.php
- If the path is static resources, index.php will decline static file requests.
- Otherwise, index.php will match the URL to the corresponding module and use the router information of that module to initial the application.
- Then, the controller take the parameters from URL can prepare the data through the Model, render the data through View and finally return the results.

<https://docs.zendframework.com/>

Zend framework – bad sample

```
index.php x bad.php x
1 <?php
2
3 namespace Application\Controller;
4
5 use Laminas\Mvc\Controller\AbstractActionController;
6 use Laminas\View\Model\ViewModel;
7 use Laminas\Db\Adapter\Adapter;
8
9 class BadController extends AbstractActionController
10 {
11     protected $adapter;
12
13     public function __construct(Adapter $adapter)
14     {
15         $this->adapter = $adapter;
16     }
17
18     public function indexAction()
19     {
20         $sql = 'SELECT * FROM my_table';
21         $resultSet = $this->adapter->query($sql, Adapter::QUERY_MODE_EXECUTE);
22
23         return new ViewModel([
24             'data' => $resultSet,
25         ]);
26     }
27 }
```

- MVC is ONLY an **architecture** helps to **split** applications into logical units.
- Frameworks makes the scaffolding ready.
- There is no restriction to write the database interaction in any framework. You may see the bad sample as shown.
- Such code violate the principle of MVC and make the project difficult to maintain in the future.

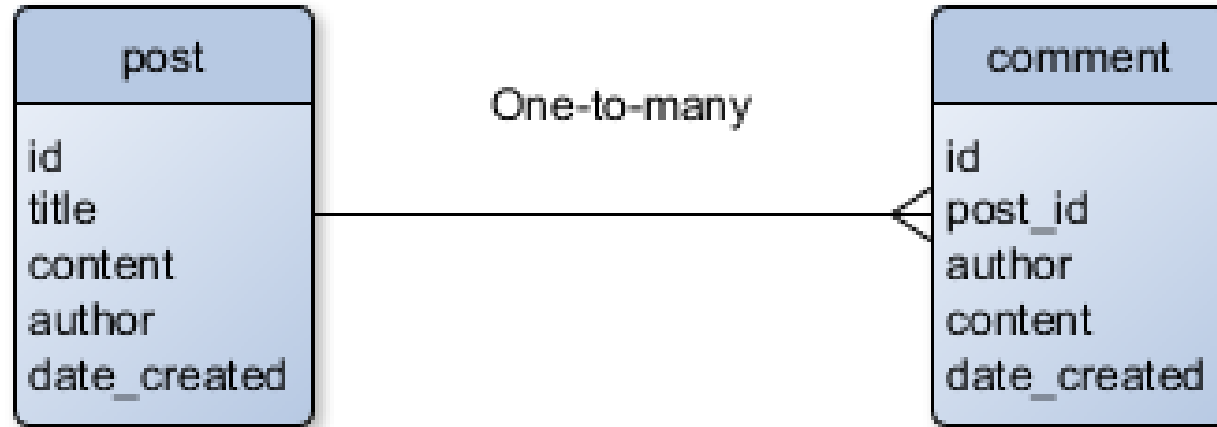
Zend framework – More Model

Model Type	Directory
Entities	APP_DIR/module/Application/src/Entity
Repositories	APP_DIR/module/Application/src/Repository
Value Objects	APP_DIR/module/Application/src/ValueObject
Services	APP_DIR/module/Application/src/Service
Factories	In Factory subdirectory under each model type directory. For example, controller factories would be stored in APP_DIR/module/Application/src/Controller/Factory

- A *model* is a PHP class which contains the business logic of your application.
- By convention (OO paradigm), models can be further subdivided into the above principal types

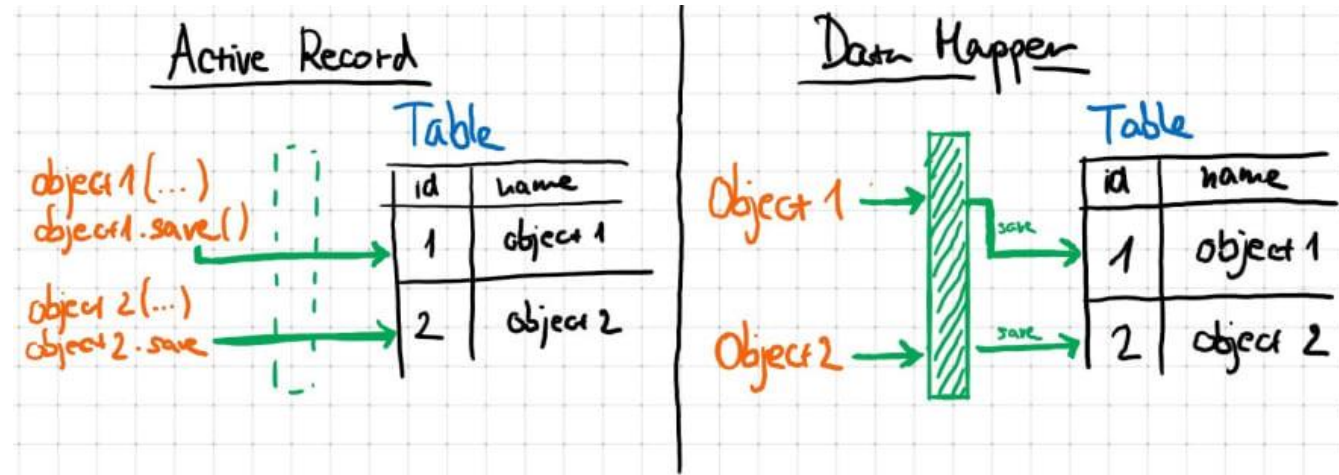
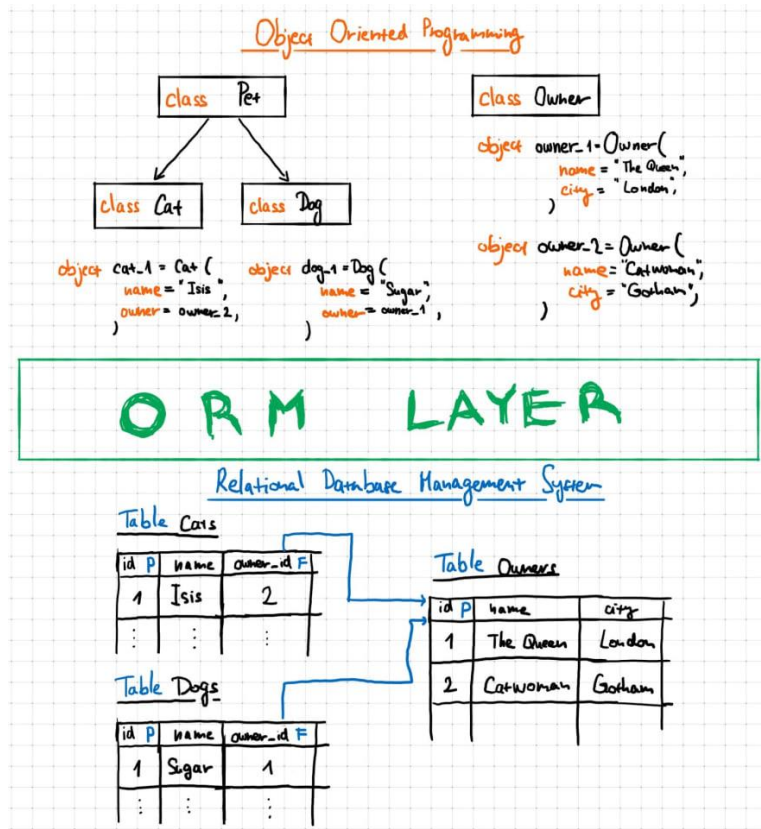
```
1 class MoneyAmount
2 {
3     // Properties
4     private $currency;
5     private $amount;
6
7     // Constructor
8     public function __construct($amount, $currency='USD')
9     {
10         $this->amount = $amount;
11         $this->currency = $currency;
12     }
13
14     // Gets the currency code
15     public function getCurrency()
16     {
17         return $this->currency;
18     }
19
20     // Gets the money amount
21     public function getAmount()
22     {
23         return $this->amount;
24     }
25 }
```

Relational of objects



- Objects have different kinds of relational with each other.
- There relational also need to be represented as objects.

ORM - Object Relational Mapping



- So many terms: **ORM DAO DataMapper ActiveRecord TableGateway**
- They are describing from different prospect.
- It is hard to explain without samples.
- But samples are beyond this module.

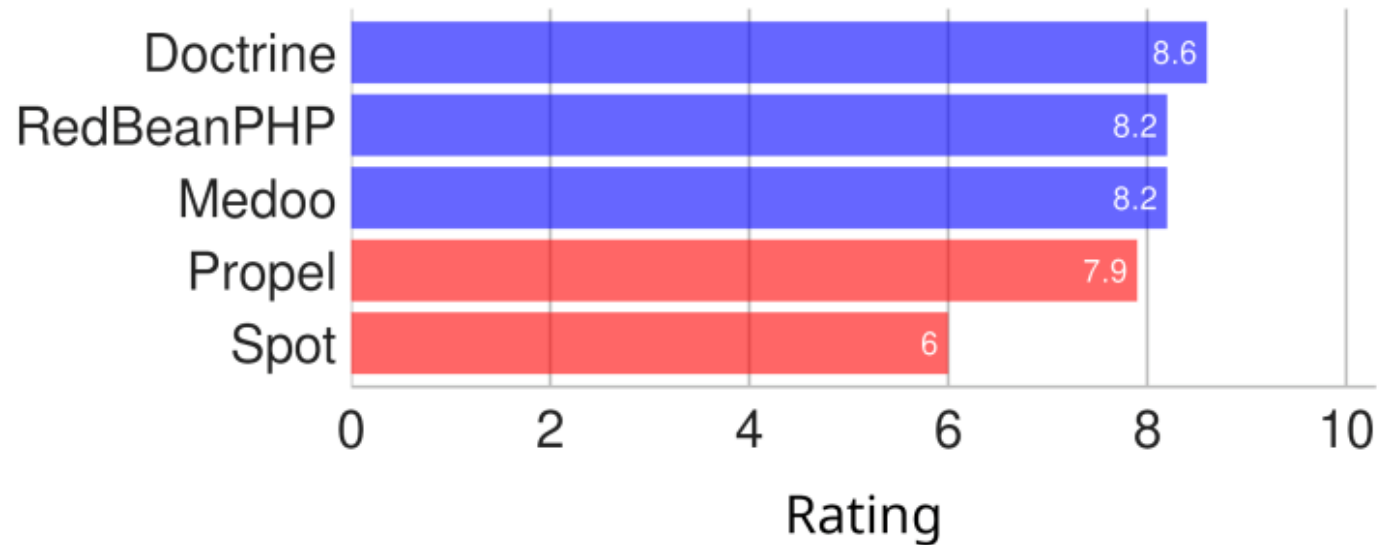
<https://dev.to/tinazhouhui/introduction-to-object-relational-mapping-the-what-why-when-and-how-of-orm-nb2>

<https://www.sitepoint.com/community/t/dao-vs-orm-vs-activerecord-vs-tablegateway-vs-ahhhh/2473>

Popular PHP ORM

Best Free PHP ORM

■ Recommended ■ Good



- So many choices...
- Familiar with each of them means a lot of coding time.

<https://www.linuxlinks.com/best-free-open-source-object-relational-mapping-software-php/>

Popular PHP frameworks



ThinkPHP®



Laravel



Symfony



CodeIgniter
CodeIgniter



Zend Framework



FuelPHP



Slim



Phalcon



Yii 2



CakePHP

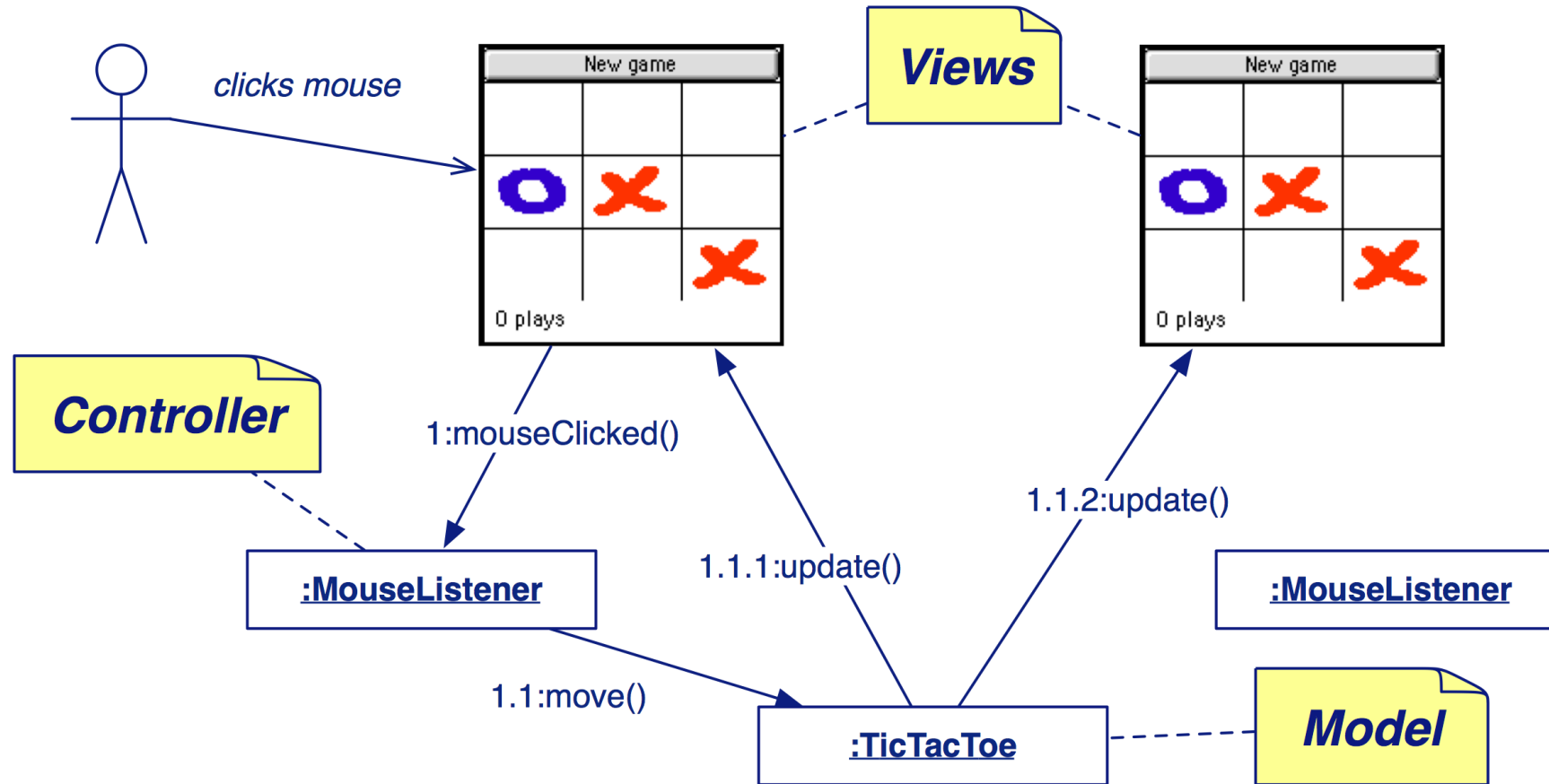
- So many choices for PHP also ...
- Familiar with each of them means a lot of coding time.

And their combination



- Actually, each language is not so complicate.
- But the eco-system and frameworks are the main challenge for a beginner.
- The frameworks are similar in principles but have their own advantages and disadvantages.
- The best practice is learn all **principles** but **NOT** the code segment in one framework.

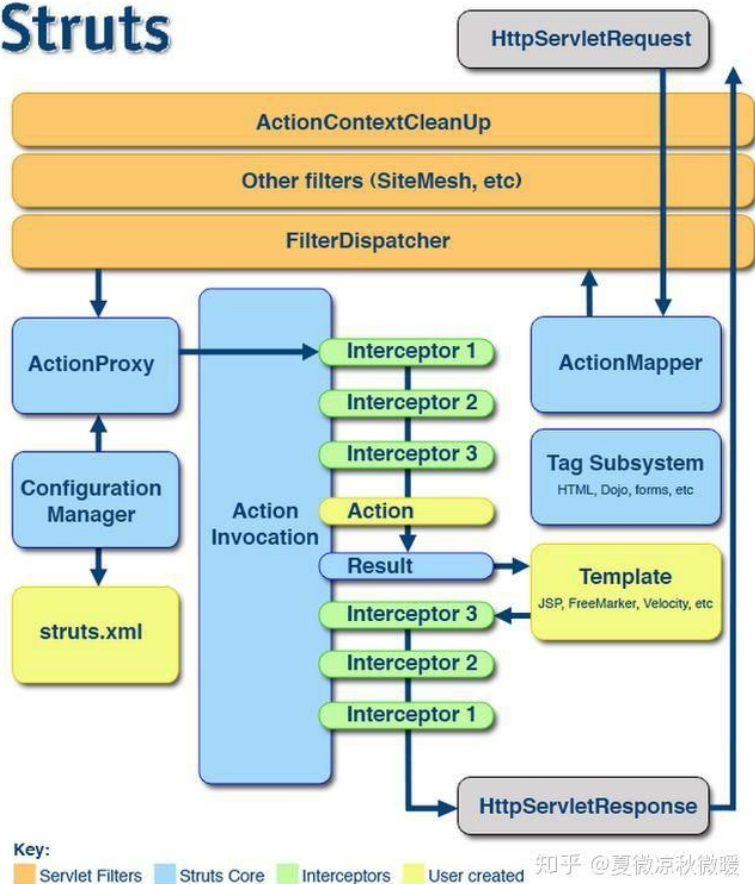
MVC & **Object** Oriented



- You may find MVC is native Object Oriented.
- JAVA is the **CHOSEN** OOP and that's why JAVA is so popular for web development.

SSH and SSM

Struts



SSH: *Spring*, Struts2 and Hibernate

<https://zhuanlan.zhihu.com/p/44149566>
<https://www.zhihu.com/question/22014461>

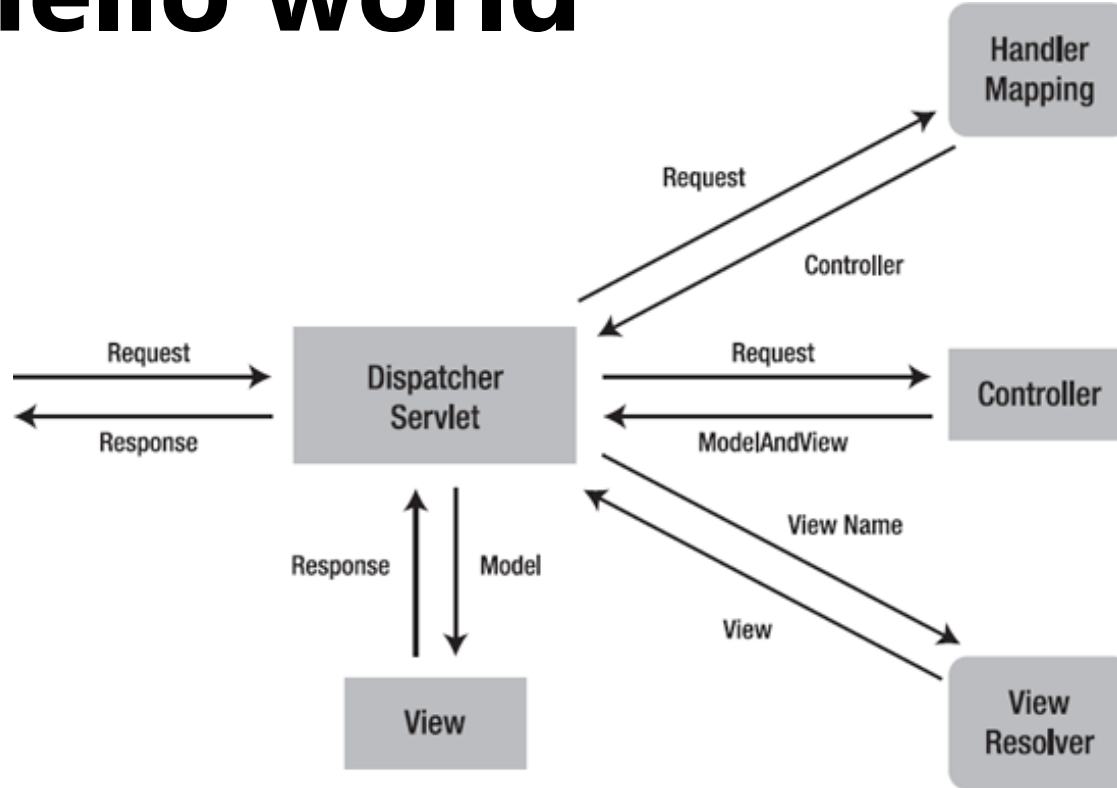
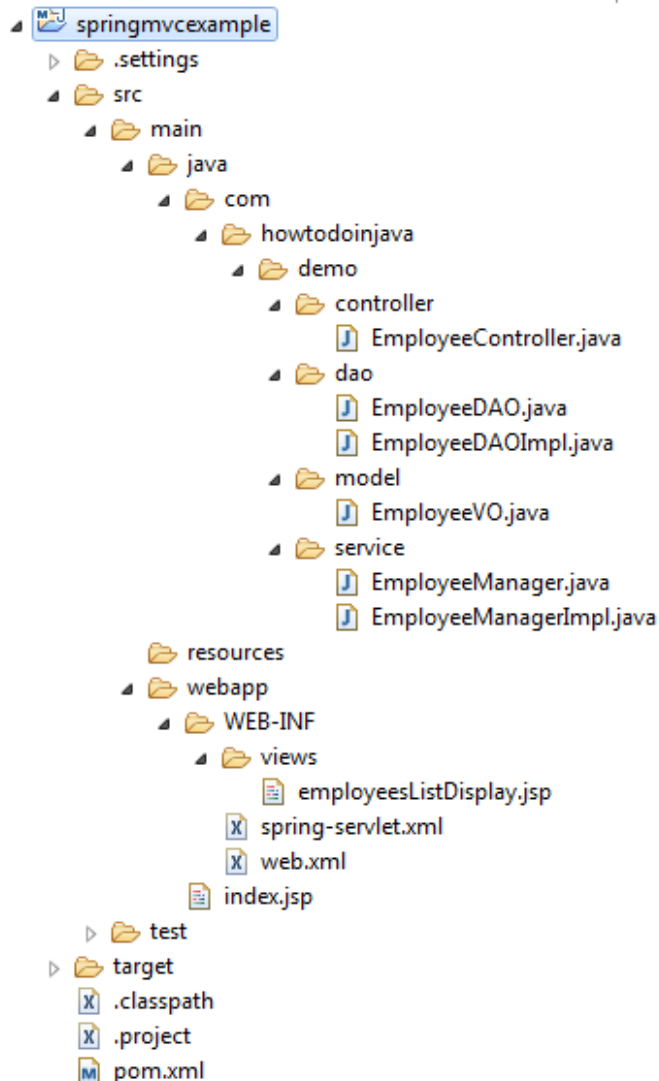
JSP is used for view layer.



知乎 @夏微凉秋微暖

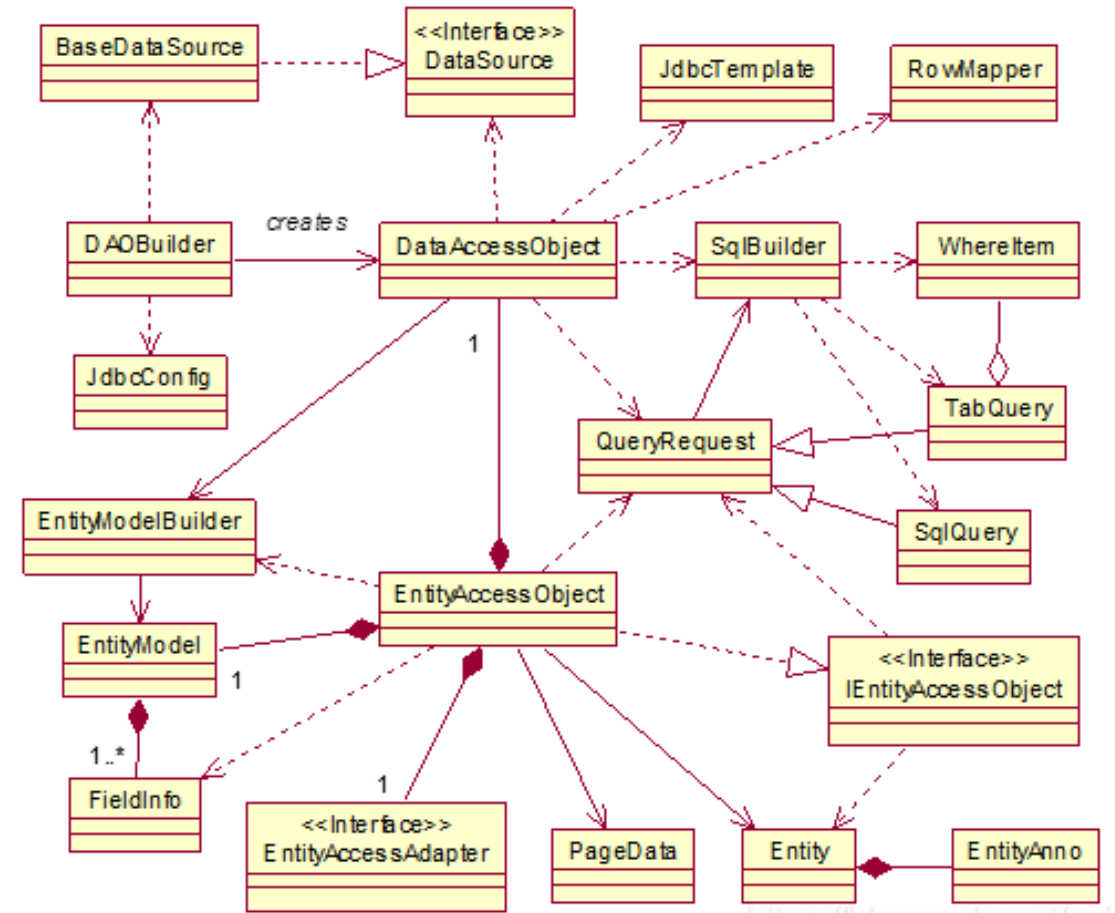
SSM: *Spring*, SpringMVC and Mybatis

Spring MVC – “Hello world”



- MVC is only a guideline to split components.
- In Spring MVC, there is a central controller, every request must go to controller and find its corresponding route.
- Understand the router and URI

Popular JAVA ORM



- So many tools...
- It means...

<https://www.tpointtech.com/orm-tools-in-java>

Hibernate and Mybatis



HIBERNATE

VS



MYBATIS

- General CRUD(CREATE, READ, UPDATE, and DELETE) functionality.
- The environment is driven by object models and needs to generate SQL **automatically**.
- Session management.

- Analytic fetch queries.
- Stored procedures and **dynamic** SQL.
- Support **complicated search queries**, where search criteria are dynamic, and paging of results.

<https://www.performatix.com/hibernate-vs-mybatis-java-development-company/>
<https://www.zhihu.com/question/309662829>

Python MVC frameworks



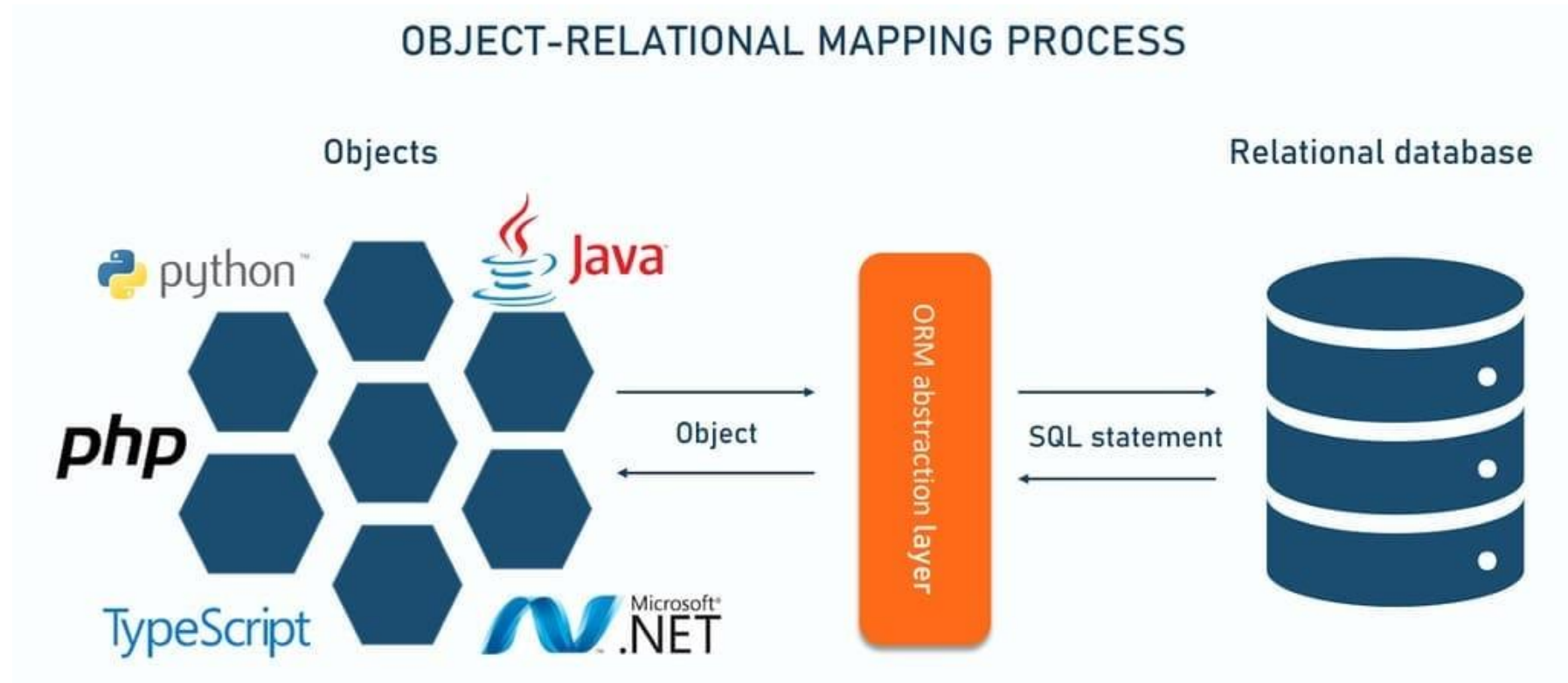
PYTHON ORM TOOLS				
	Best fit for	Database support	Main pros	Main cons
django	Django web development	✓ PostgreSQL ✓ MySQL/MariaDB ✓ Oracle ✓ SQLite	✓ Intuitive, Pythonic way to interact with databases ✓ Out-of-the-box data schema migration	✓ Difficulties with writing complex queries ✓ Additional effort needed for using outside of Django
SQLAlchemy	Enterprise-grade non-Django projects	✓ PostgreSQL ✓ MySQL/MariaDB ✓ Oracle ✓ SQLite ✓ MS-SQL Server ✓ Firebird	✓ Flexible ✓ Control over SQL ✓ Better performance with complex queries	✓ High complexity for users new to ORM ✓ Small community to find support from
PEELER	Small and medium-sized projects	✓ PostgreSQL ✓ MySQL ✓ SQLite ✓ CockroachDB	✓ Lightweight ✓ Easy to install and use ✓ Support for advanced queries	✓ Small ecosystem of third-party integrations ✓ Small community ✓ No automated data schema migration
Pony	Startups, researchers	✓ PostgreSQL ✓ MySQL ✓ SQLite ✓ Oracle ✓ CockroachDB	✓ Convenient syntax for writing queries ✓ Automatic query optimization ✓ Database schema editor	✓ Small community ✓ No automated data schema migration

- Python also has the eco-society for WEB MVC and ORM

<https://blog.polosoftech.com/python-development-a-key-to-success-for-your-business>

<https://www.altexsoft.com/blog/object-relational-mapping-tools/>

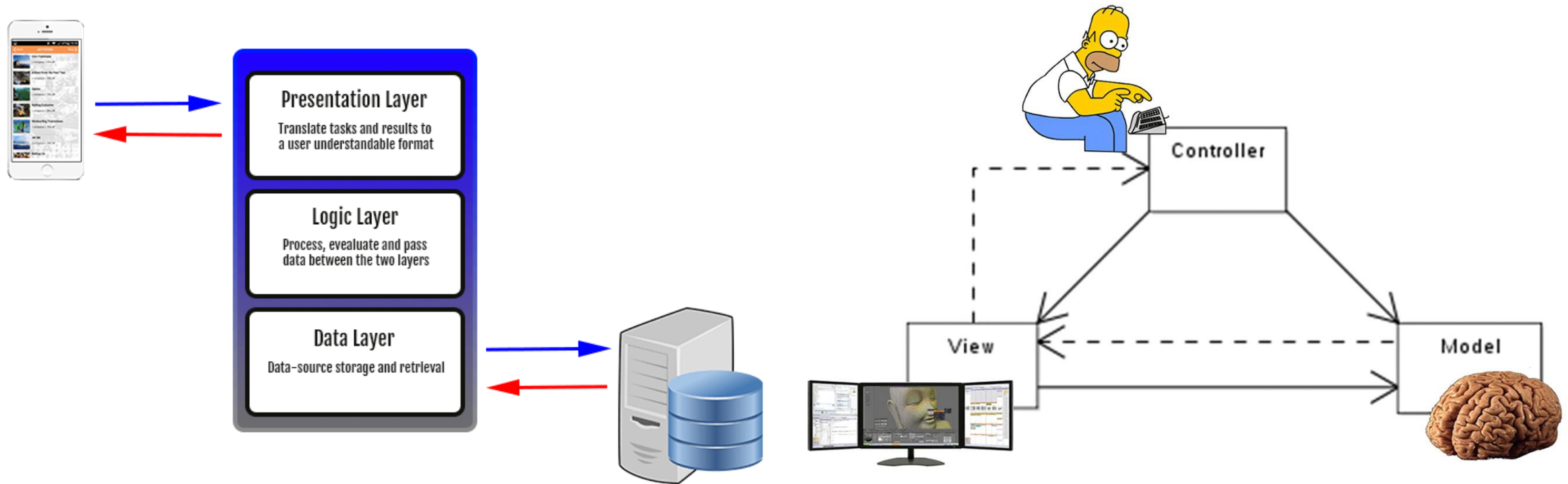
How to chose?



- It is an open question with no given answer.
- All of them have success stories.
- Pay the cost (time and money) in real projects and you will be a master

<https://www.altexsoft.com/blog/object-relational-mapping-tools/>

Three layers and MVC



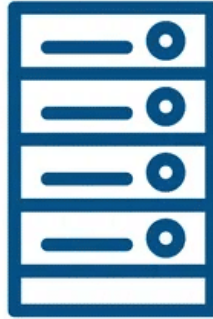
- Can you tell the difference between these two?

Split **the** web project



Front End

- Markup and web languages such as HTML, CSS and Javascript
- Asynchronous requests and Ajax
- Specialized web editing software
- Image editing
- Accessibility
- Cross-browser issues
- Search engine optimisation

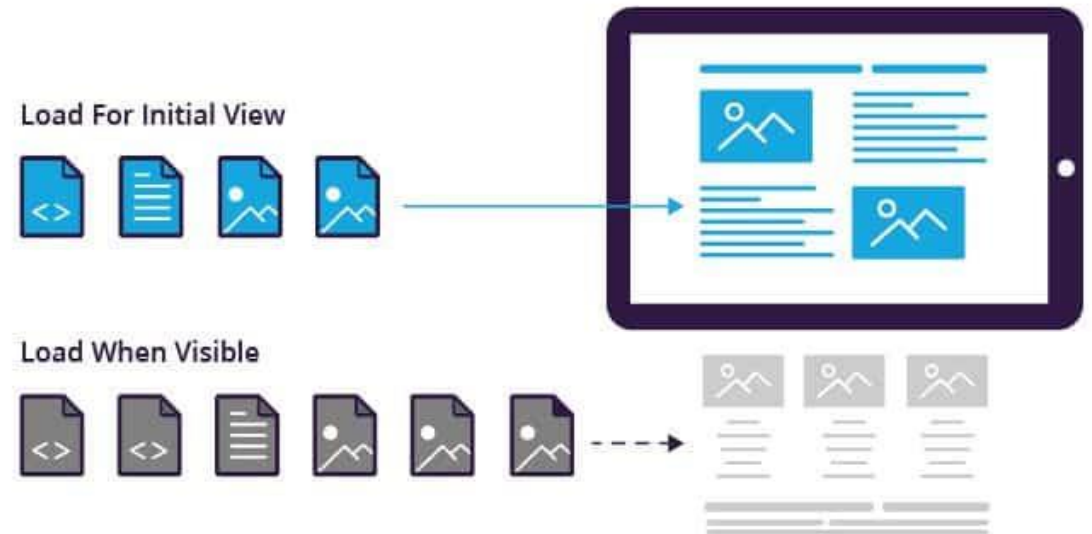
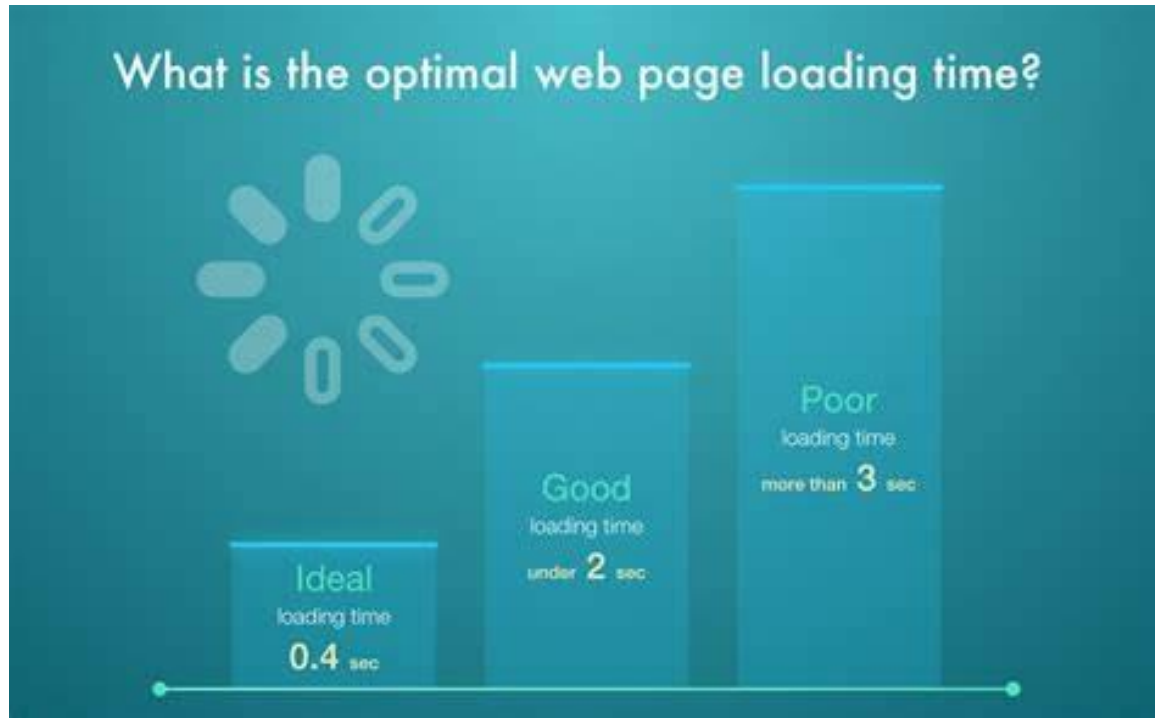


Back End

- Programming and scripting such as Python, Ruby and/or Perl
- Server architecture
- Database administration
- Scalability
- Security
- Data transformation
- Backup

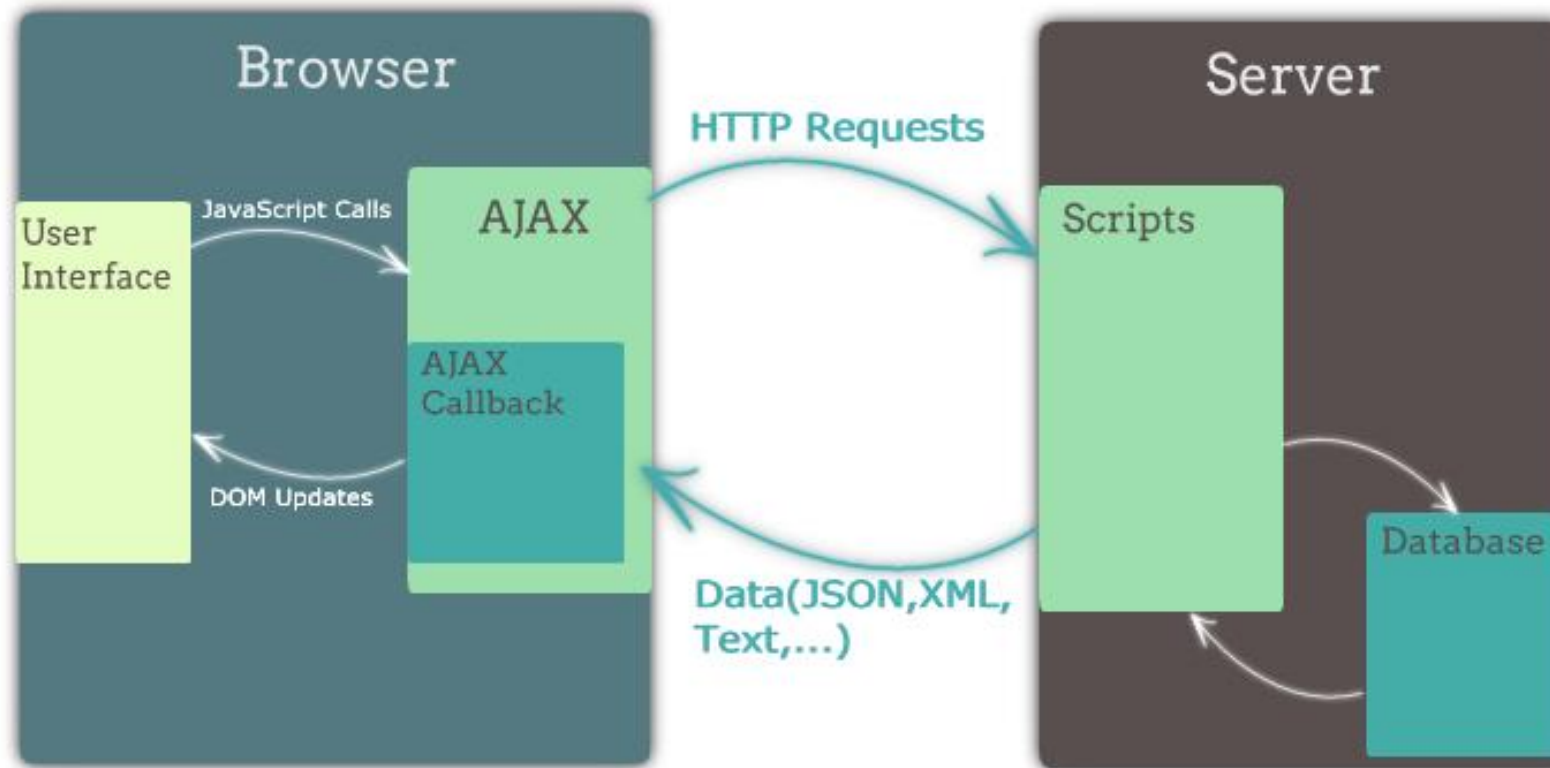


Load the huge resources



- Web content is more and more complicate.
- It means more resource to be loaded and longer loading time.
- The solution is loading/updating on demand.

Web - **AJAX** (**A**synchronous **J**avaScript **A**nd **X**ML)



- Ajax enable browser (front-end) to interact with web server (back-end) within the same webpage.
- Ajax can be applied to make dynamic front-end developing.

<https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX>

A sample page with jQuery

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>jQuery AJAX example</title>
  </head>
  <body>    droplist of id="states"
    <select id="states">
      <option>AK</option>
      <option>HI</option>
      <option>NY</option>
      <option>TX</option>
    </select>
    <p>It's capital is <span id="capital"></span></p>

    <script src="http://codeorigin.jquery.com/jquery-2.0.3.min.js">
  </script>
  <script>
    $(document).ready(function() {

    });
  </script>
</body>
</html>
```

```
<script>
$(document).ready(function() {
    $('#states').change(function() {
        selectedState = $("#states option:selected").text();

        $.get('compute capitals.php?state='+selectedState,
function( data ) {
            $('#capital').html( data );
        });
    });
});
</script>
```

The selected state in droplist: AK, HI, TX, etc

pass the URL

pass the callback function which does this with the retrieved data

AJAX call

- jQuery is a JS project.
- It likes an SDK which has more functions than Ajax.
- Bootstrap includes jQuery

<https://learnwebtutorials.com/jquery-ajax-example-and-tutorial>

XML - Extensible Markup Language

XML File Structure

Internal File Name of your choosing which marks the beginning of the File Data.



```
<?xml version="1.0" encoding="UTF-8"?>
```



Header Record so that processing Apps know it is an XML File.

```
<Records>
```

```
<Player>
```

```
<Name>Aldin Setkic</Name>
```

```
<Country>Bosnia-Herzegovina</Country>
```

```
<BirthDate>21-12-1987</BirthDate>
```

```
</Player>
```

Record Name of your own choosing which marks the start of this record.



```
<Player>
```

```
<Name>Anna Blinkova</Name>
```

```
<Country>Russia</Country>
```

```
<BirthDate>10-09-1998</BirthDate>
```



A Field within the record.
All fields are Plain Text Type by default.

End of record Tag with a /



```
</Player>
```

```
<Player>
```

```
<Name>Bernada Pera</Name>
```

```
<Country>United States</Country>
```

```
<BirthDate>03-12-1994</BirthDate>
```

```
</Player>
```

```
<Player>
```

```
<Name>John Patrick Smith</Name>
```

```
<Country>Australia</Country>
```

```
<BirthDate>24-01-1989</BirthDate>
```

```
</Player>
```

File Name with a / in front of it to indicate the End of this File.



```
</Records>
```



There is no Footer - when the chars stop it is end of the file.

The App or Software reading the XML file knows the file hierarchy structure, and can identify the start <....> and end </....> tags for the File, and its Records, and Fields.

- XML extend HTML tags, used for the request/response of AJAX

<https://passyworldofict.com/programming/programming22/>

XML → JSON (JavaScript Object Notation)

```
<employees>
  <employee>
    <firstName>Bill</firstName>
    <lastName>Gates</lastName>
  </employee>
  <employee>
    <firstName>Steve</firstName>
    <lastName>Jobs</lastName>
  </employee>
  <employee>
    <firstName>Elon</firstName>
    <lastName>Musk</lastName>
  </employee>
</employees>
```

```
{"employees":[
  { "firstName":"Bill", "lastName":"Gates" },
  { "firstName":"Steve", "lastName":"Jobs" },
  { "firstName":"Elon", "lastName":"Musk" }
]}
```

- Which format do you like?
- JSON is more popular.
- Most language support json translation

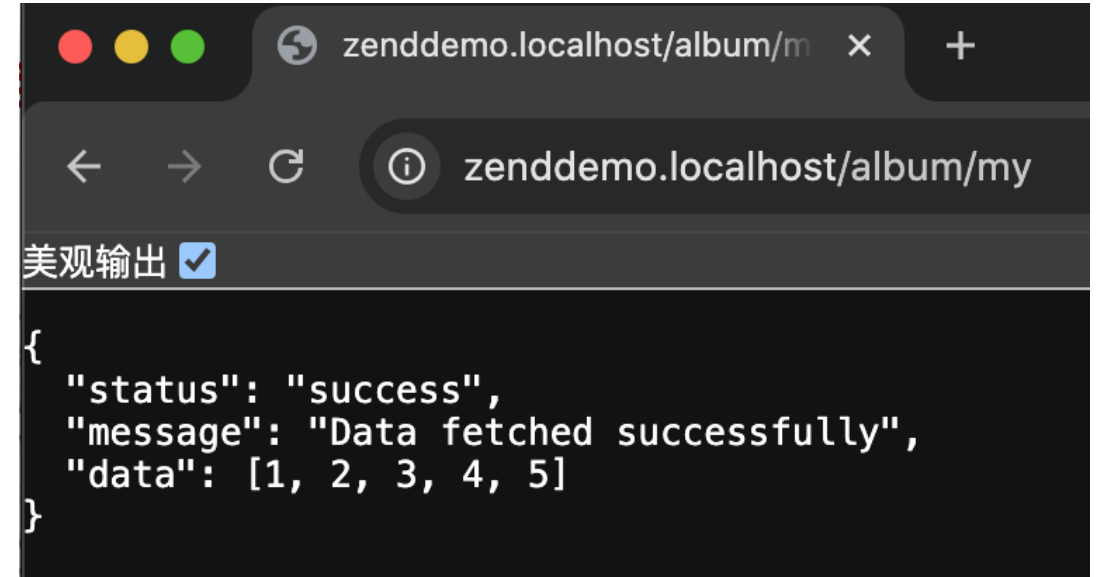
https://www.w3school.com.cn/js/js_json_xml.asp

<https://www.toptal.com/web/json-vs-xml-part-1>

<https://twobithistory.org/2017/09/21/the-rise-and-rise-of-json.html>

Zend controller for JSON

```
21 public function myAction()  
22 {  
23     $data = [  
24         'status' => 'success',  
25         'message' => 'Data fetched successfully',  
26         'data' => [1, 2, 3, 4, 5]  
27     ];  
28  
29     $this->response->getHeaders()->addHeaderLine('Content-Type', 'application/json');  
30     $this->response->setContent(json_encode($data));  
31     return $this->response;  
32 }
```



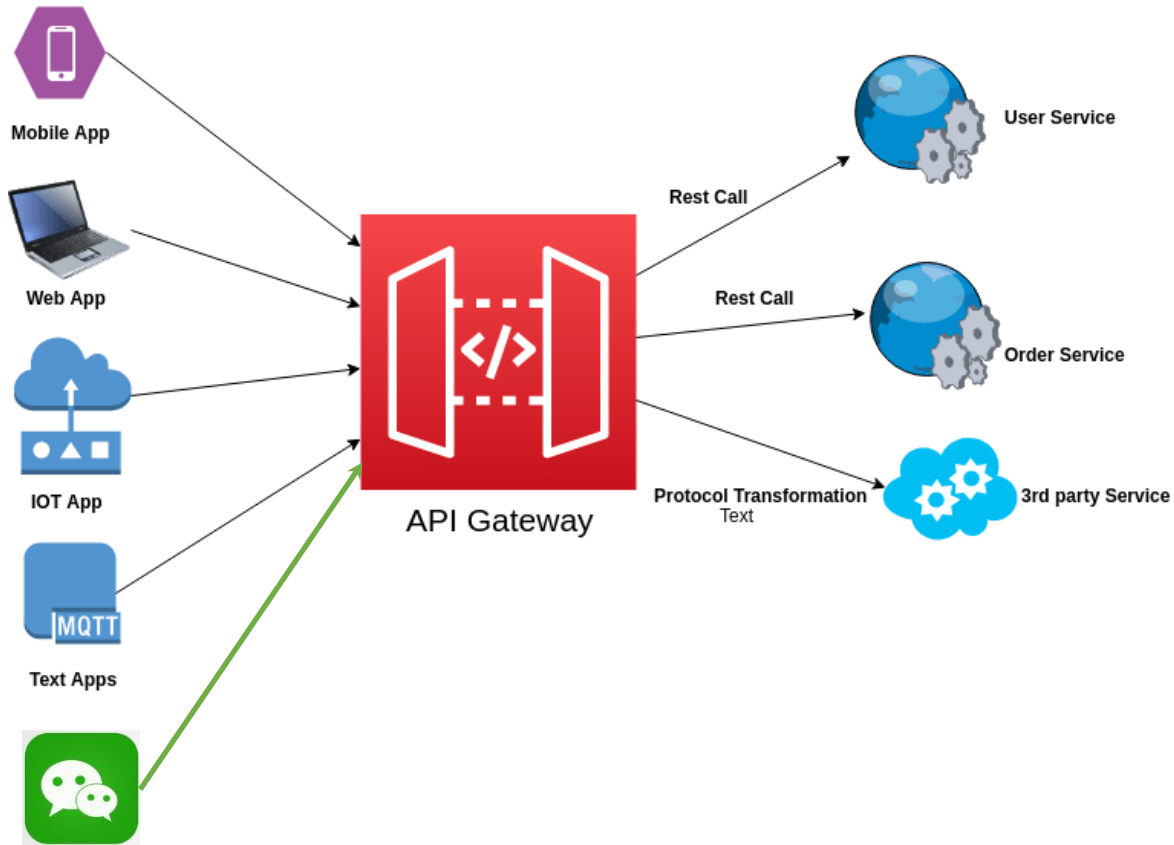
- Zend framework can response the data in JSON format.
- The above sample shows the code and corresponding output.

Front-end & Back-end split



- Since the technology stacks are very different from each other, the web project can be split into Front-end system and Back-end system.
 - The **view** is used to present the graphical visualization of the *“user”* interface.
 - JSON results can be regarded as the **View** of back-end (if MVC) **AND Model** of front-end.
- Split means front-end and back-end are **TWO individual** systems

Another approach is multi front-ends

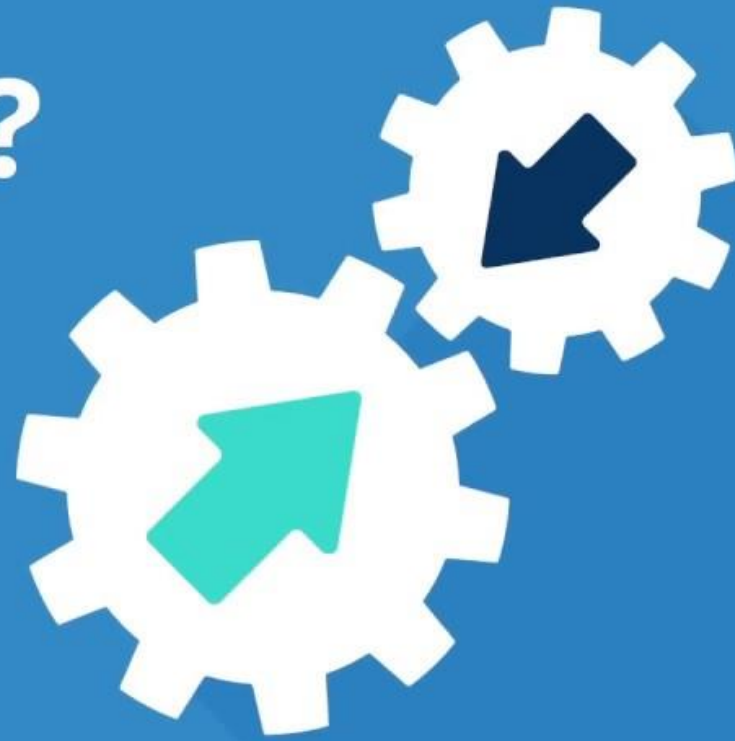


- Many new front-ends in mobile age, the main types are: PC web, mobile web, iOS/android/HarmonyOS native APP and **WeChat** application.
- Their appearance are very different from each other while back-end logic are most same.
- Let the present layer define their layouts and back-end process the data is a way to split.
- **Restful WEB APIs** can support different front-ends.

API (Application Programming Interface)

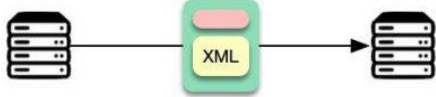
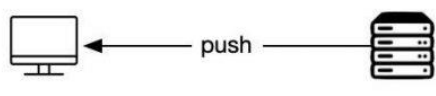
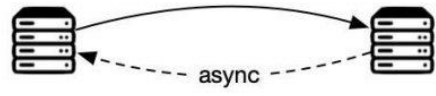
What is an API?

An API is a very useful mechanism that allows two pieces of software to exchange messages or data in a standard format. Thus, it becomes an instrument to search for new revenue streams, open the doors to talent, or automate processes in an innovative way.



API (Application Programming Interface)

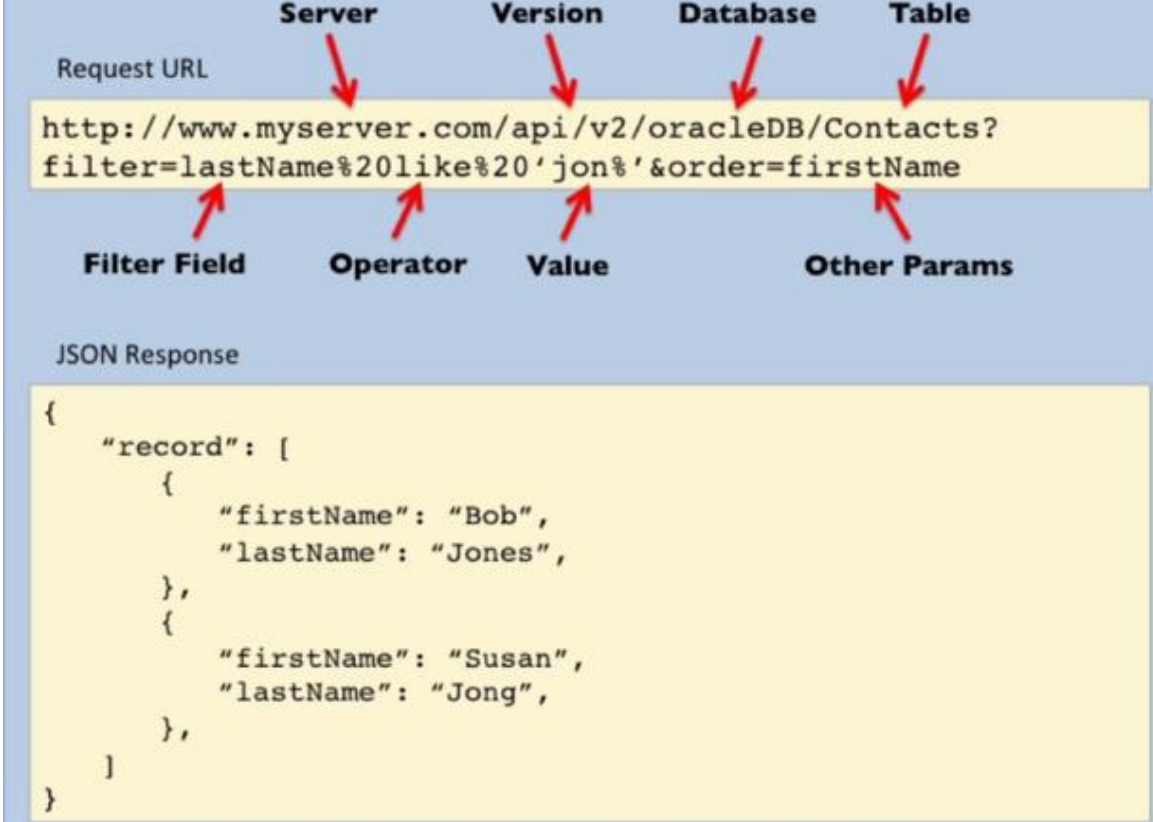
Top 6 Most Popular API Architecture Styles  ByteByteGo.com

Style	Illustration	Use Cases
SOAP		XML-based for enterprise applications
RESTful		Resource-based for web servers
GraphQL		Query language reduce network load
gRPC		High performance for microservices
WebSocket		Bi-directional for low-latency data exchange
Webhook		Asynchronous for event-driven application

API architectural styles define how different components of an application interact with each other through APIs

<https://www.devopsschool.com/blog/top-api-architectural-styles/>

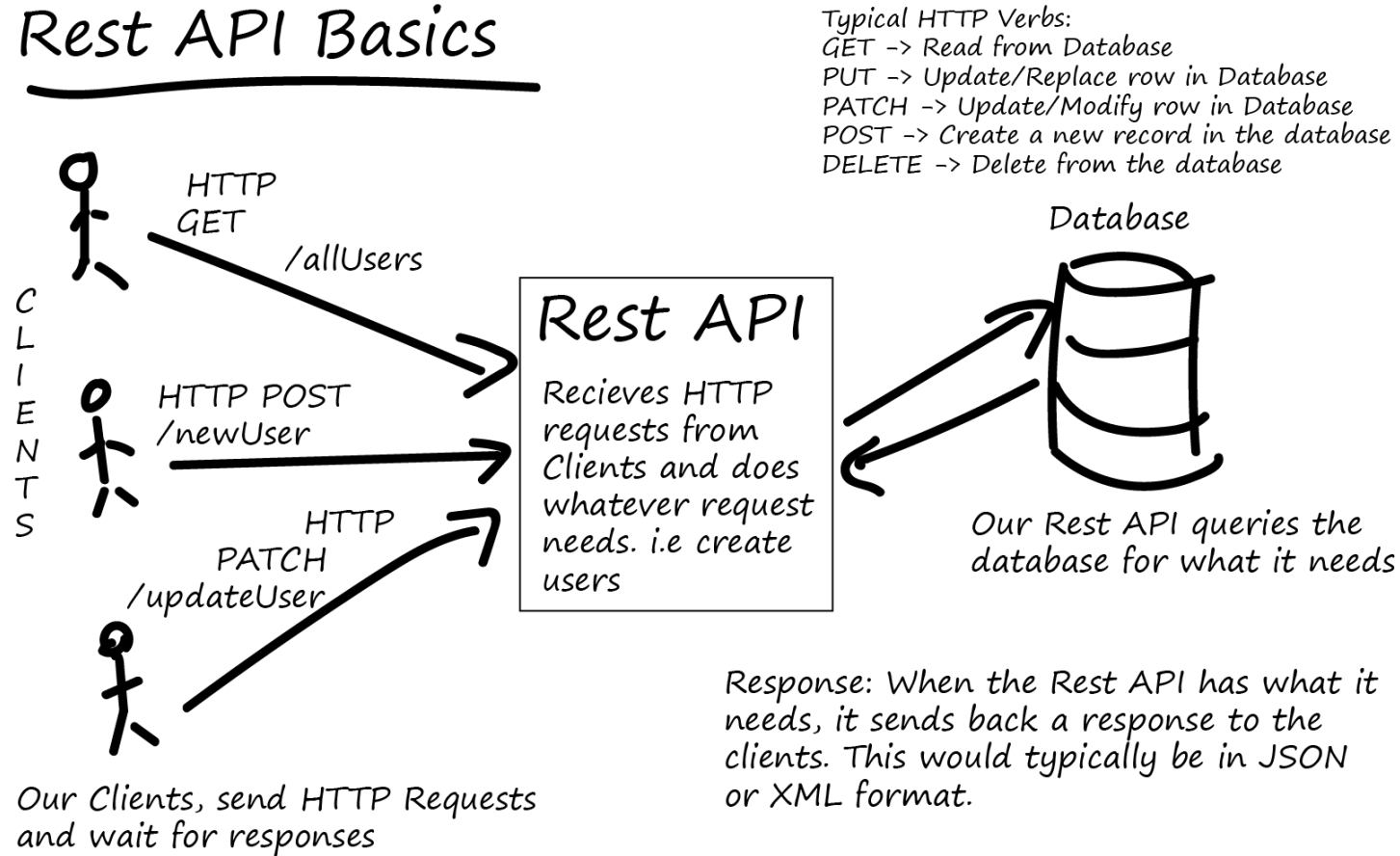
A central laptop icon with 'API' on its screen, surrounded by various tech-related icons like gears, code symbols, and network lines.



- **Two systems** exchange information over **HTTP** protocol

REST (Representational State Transfer) API

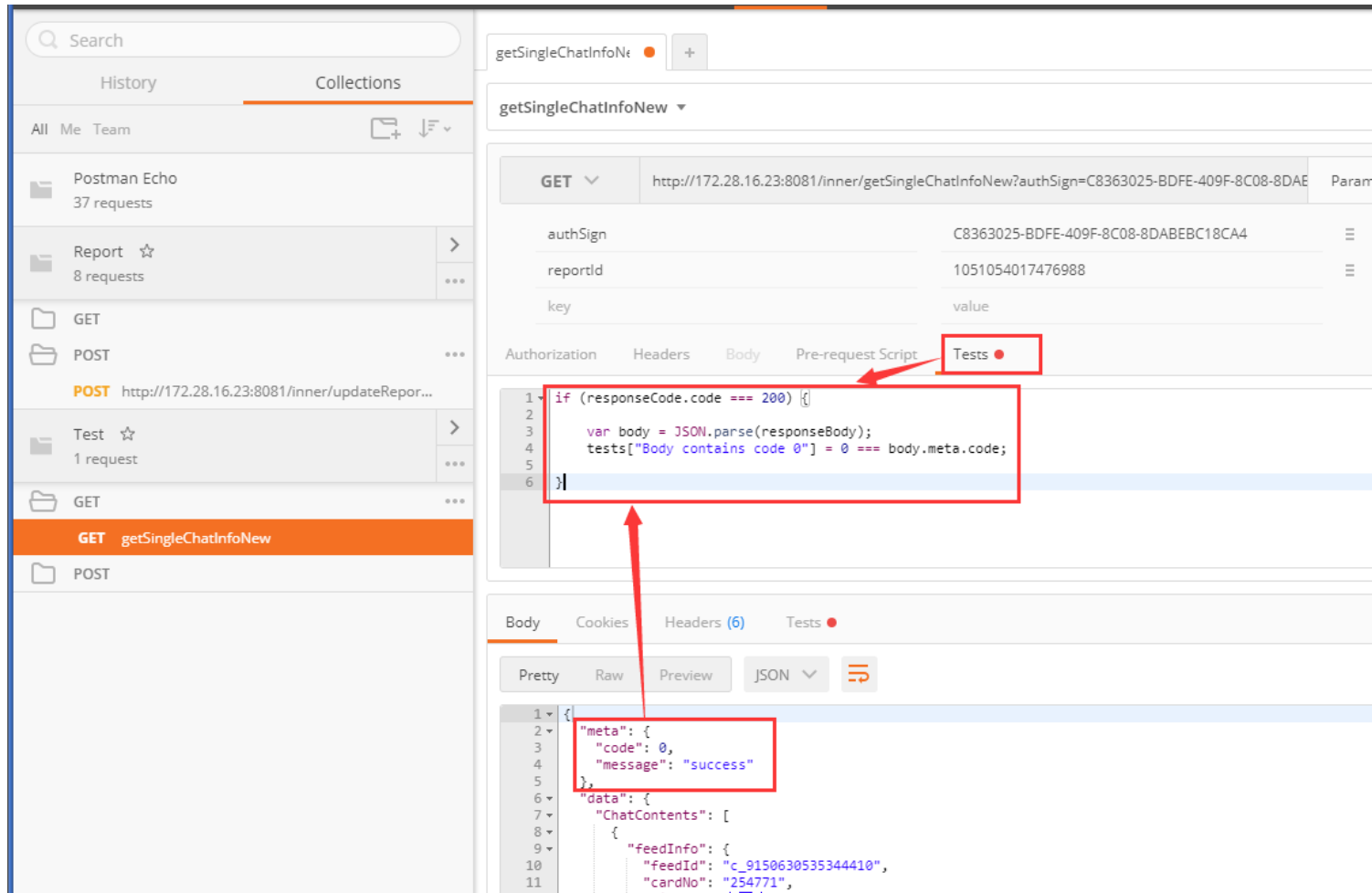
Rest API Basics



- REST APIs add **NO** new capability to HTTP APIs.
- But it is an architectural **style** that was created in tandem with HTTP

<https://tutorialedge.net/software-eng/what-is-a-rest-api/>

API test tool - Postman



- Better experience to send and resolve WEB API data

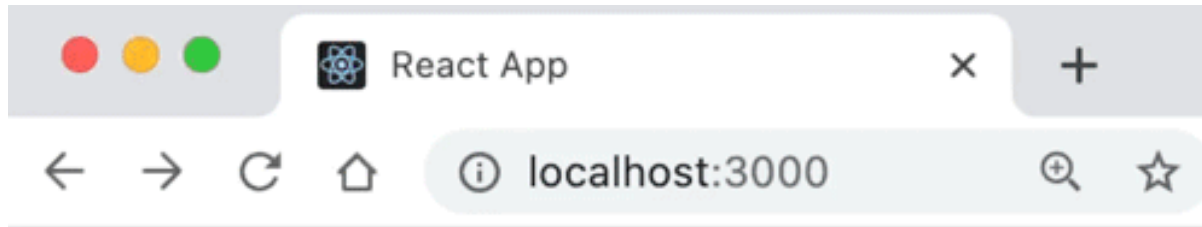
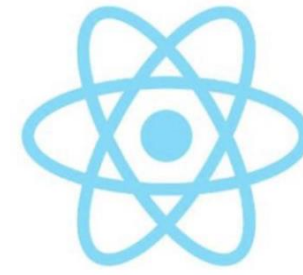
<https://www.esds.co.in/blog/manual-testing-process-lifecycle/>

Front-end evolution



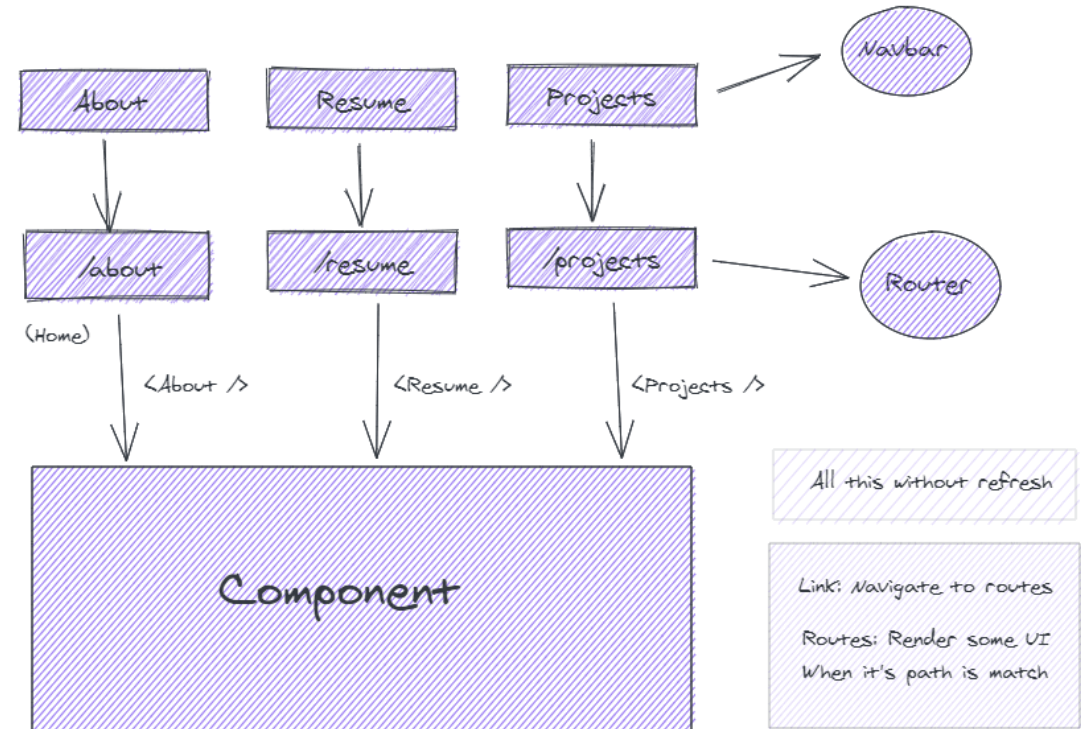
- We can generate all webpages as static resources.
- Each page for an individual file.
- It can get dynamic data with jQuery.
- But compare with MVC, let router to manage the URL mapping, which one is better?

SPA: single page application



HomeSurvey

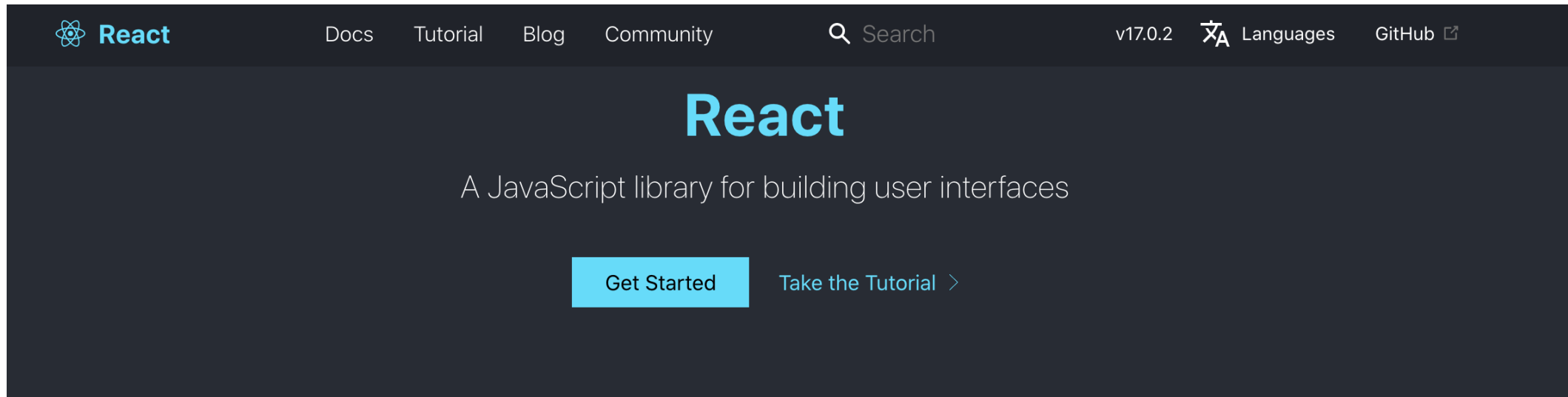
Home page



- Front-end also **can** have the **router**.
- The URI change is caught by **front-end router**.

<https://openclassrooms.com/en/courses/7315991-intermediate-react/7572037-turn-your-application-into-a-single-page-app-with-react-router>

React



Declarative

React makes it painless to create interactive UIs. Design simple views for

Component-Based

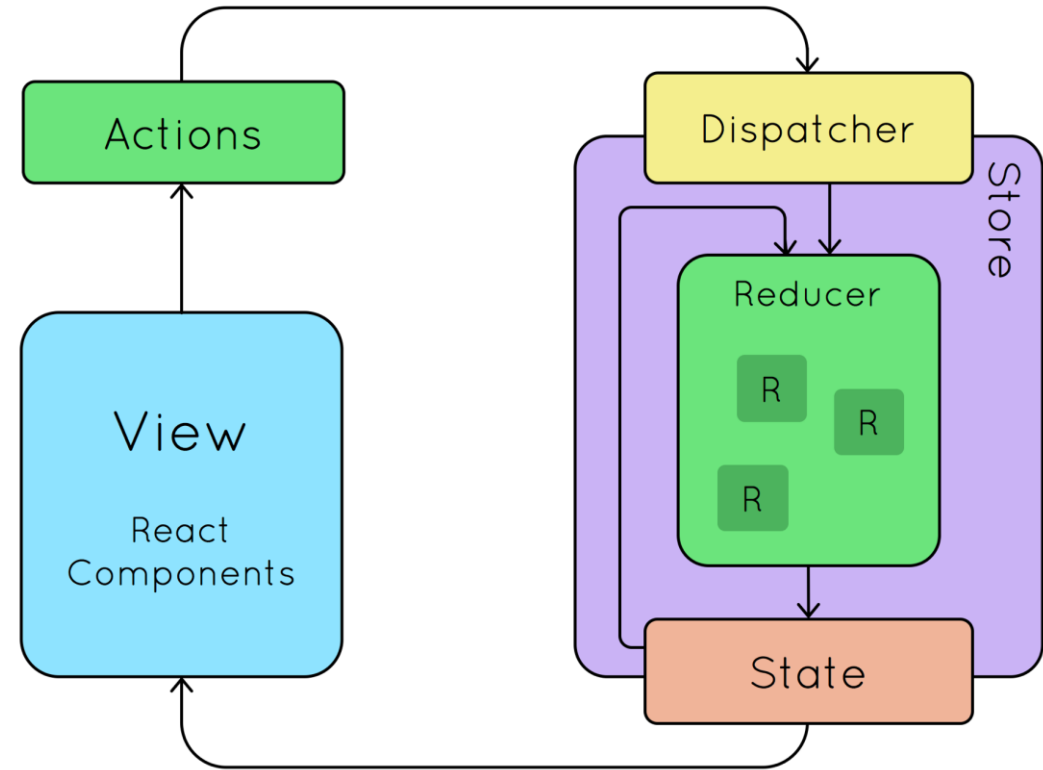
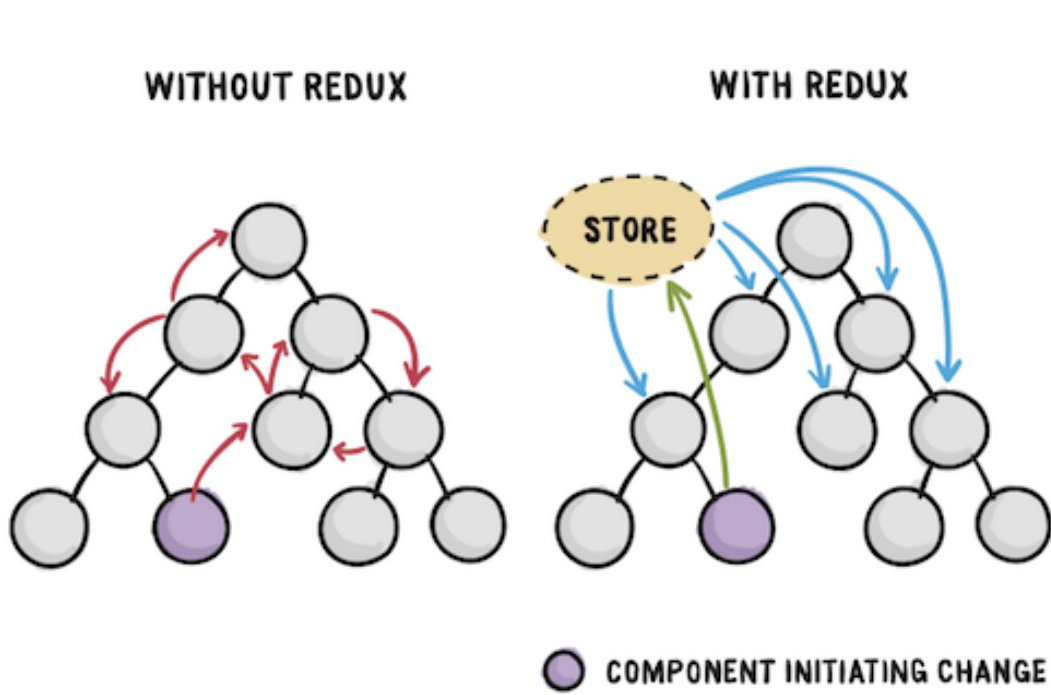
Build encapsulated components that manage their own state, then compose

Learn Once, Write Anywhere

We don't make assumptions about the rest of your technology stack, so you can

- Original by Facebook and opensource in 2013
- Using JSX language (similar to JS)

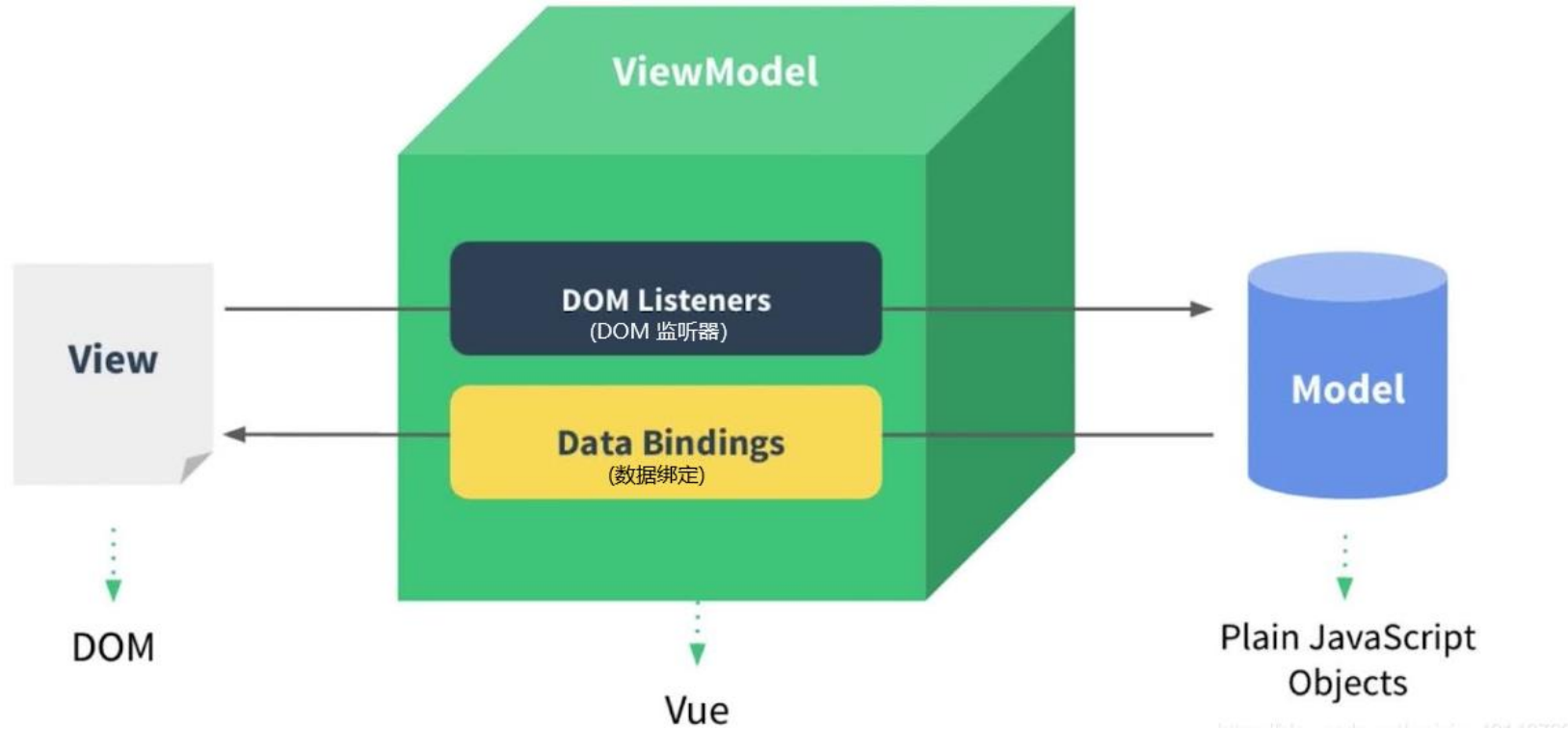
React + redux



- **React + redux can work in MVC model**
- **React does not include the router and need to install the individual router module**

<https://velog.io/@hyeon4137/ReactRedux-1>

Vue – Model-View-**ViewModel** (MV**VM**)

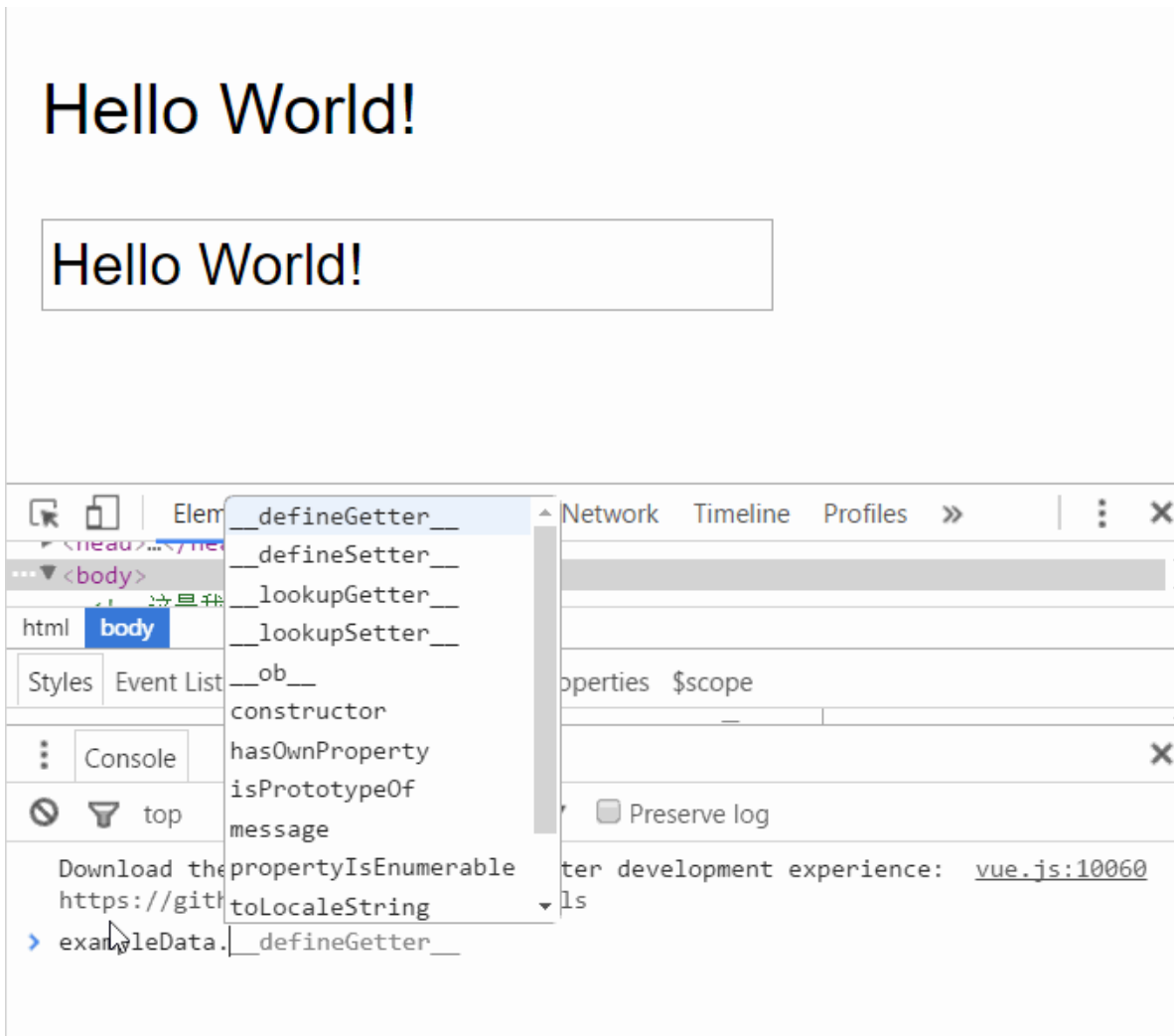
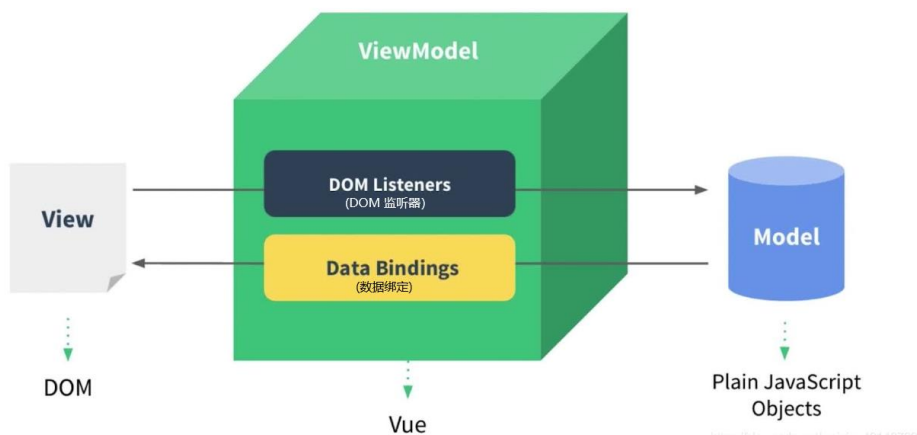


- Original by 尤雨溪 in 2014
- It is not MCV; No typical **controller**.
- Data and view are actually the same, but have different appearance

<https://blog.csdn.net/youlingxx/article/details/79361979>

<https://zhuanlan.zhihu.com/p/143876237>

Vue – two-way binding



<https://blog.csdn.net/youlingxx/article/details/79361979>

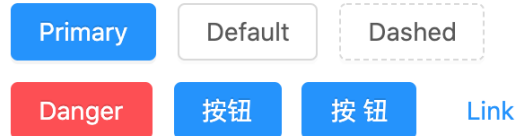
Vue – components library



Component Registration

```
import { Button } from 'ant-design-vue';  
Vue.use(Button);
```

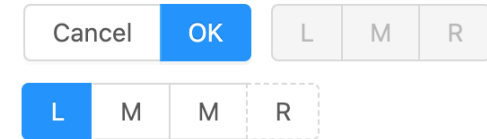
Examples



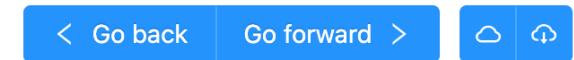
Type

There are `primary` button, `default` button, `dashed` button, `danger` button and `link` button in antd.

Basic



With Icon



- UI libraries for Vue, like Bootstrap for traditional CSS/JS

Vue: sample code

```
1  <!DOCTYPE html>
2  <html lang="">
3  <head>
4    <meta charset="utf-8">
5    <meta http-equiv="X-UA-Compatible" content="IE=edge">
6    <meta name="viewport" content="width=device-width,initial-scale=1.0">
7    <link rel="icon" href="<%= BASE_URL %>favicon.ico">
8    <title><%= htmlWebpackPlugin.options.title %></title>
9  </head>
10 <body>
11   <noscript>
12     <strong>We're sorry but <%= htmlWebpackPlugin.options.title %> doesn't work properly without J
13   </noscript>
14   <div id="app"></div>
15   <!-- built files will be auto injected -->
16 </body>
17 </html>
18
```

- Only one "index.html"

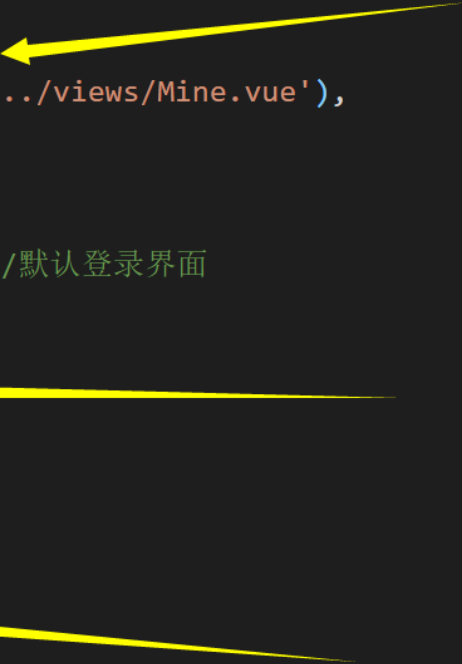
Vue: sample code

```
1  <template>
2    <div class="hello">
3      <h1>{{ msg }}</h1>
4      <h3>
5        It's my Vue now!
6      </h3>
7    </div>
8  </template>
9
10 <script>
11   export default {name: 'HelloWorld'...}
17 </script>
18
19   <!-- Add "scoped" attribute to limit CSS to this component only -->
20 <style scoped>
21   h3 {...}
24   ul {
25     list-style-type: none;
26     padding: 0;
27   }
```

- Are you familiar with it?

Vue: front-end router

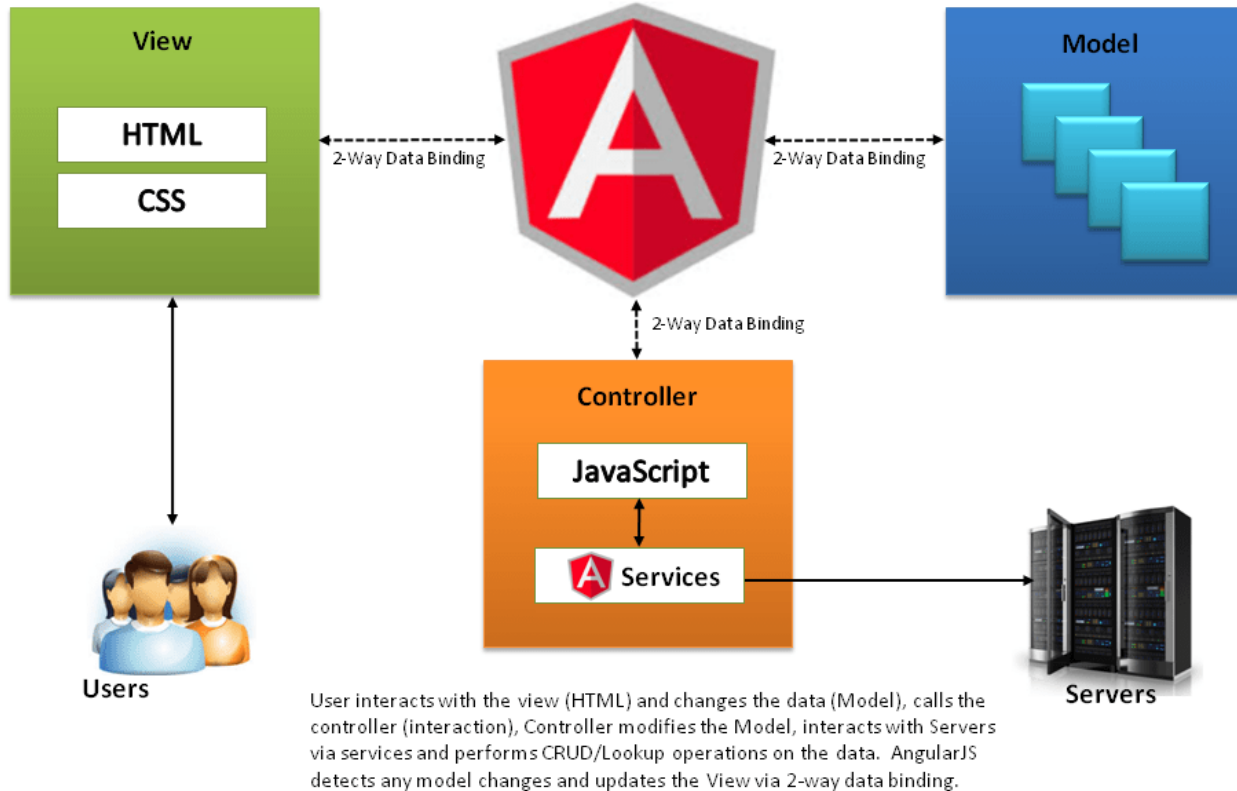
```
51 },
52 {
53   path: '/mine',
54   name: 'mine',
55   component: () => import('../views/Mine.vue'),
56   children: [
57     {
58       path: '',
59       component: Login //默认登录界面
60     },
61     {
62       path: 'login',
63       name: 'login',
64       component: Login
65     },
66     {
67       path: 'reg',
68       name: 'reg',
69       component: Reg
70     }
71   ]
72 },
```



- Capture the path by **browser router** and change the corresponding component without loading a new webpage

<http://www.wangt.cc/2021/08/vue-router-2/>

Angular: fully MVC



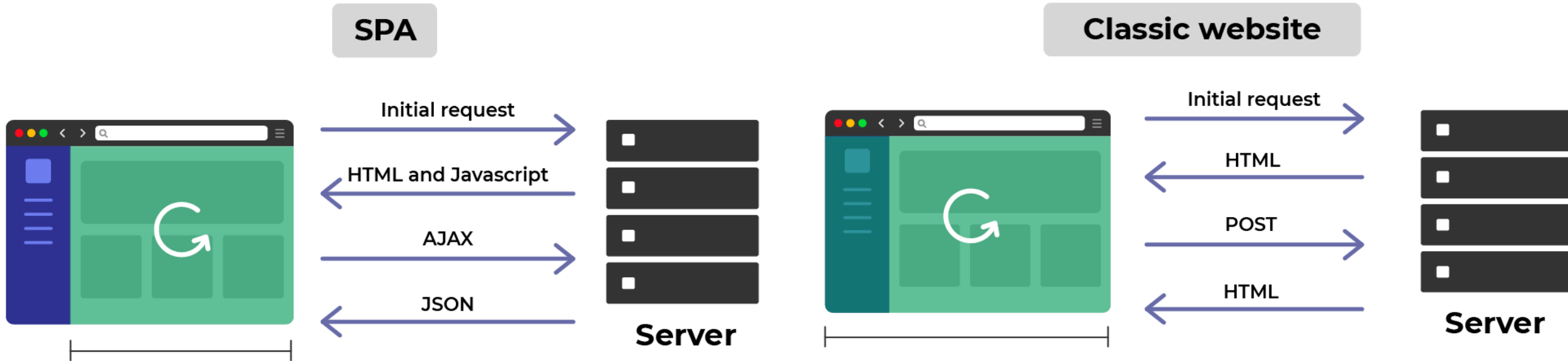
- Original by Google in 2016
- Official router, animation, SSR, e2e

<https://angular.io/docs>

<https://www.zhihu.com/question/383137600/answer/2401279819>

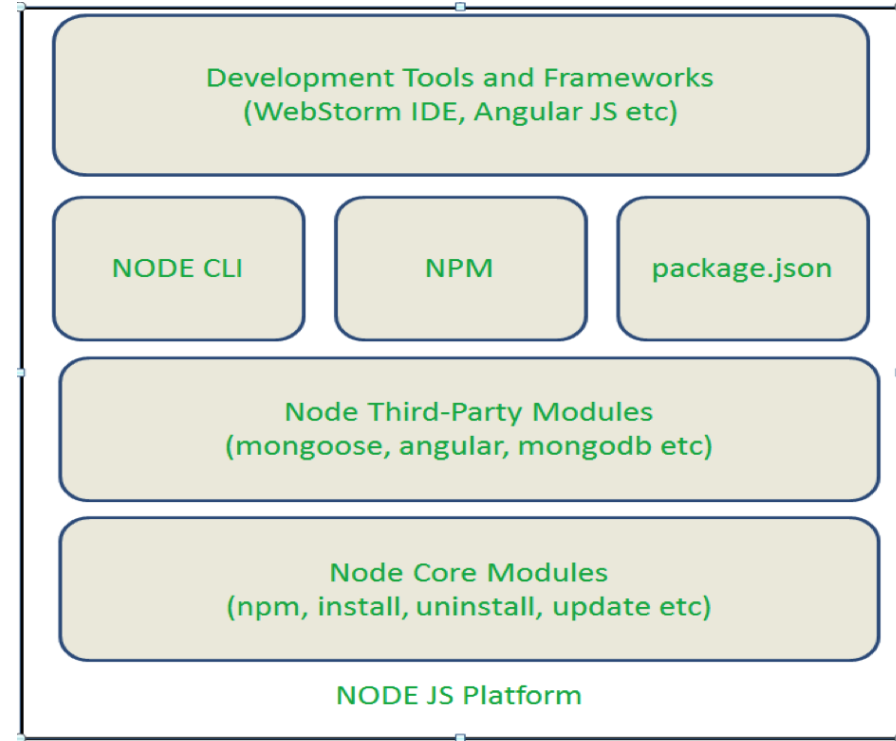
<https://blog.csdn.net/GUDUzhongliang/article/details/129747078>

SPA vs. classic website



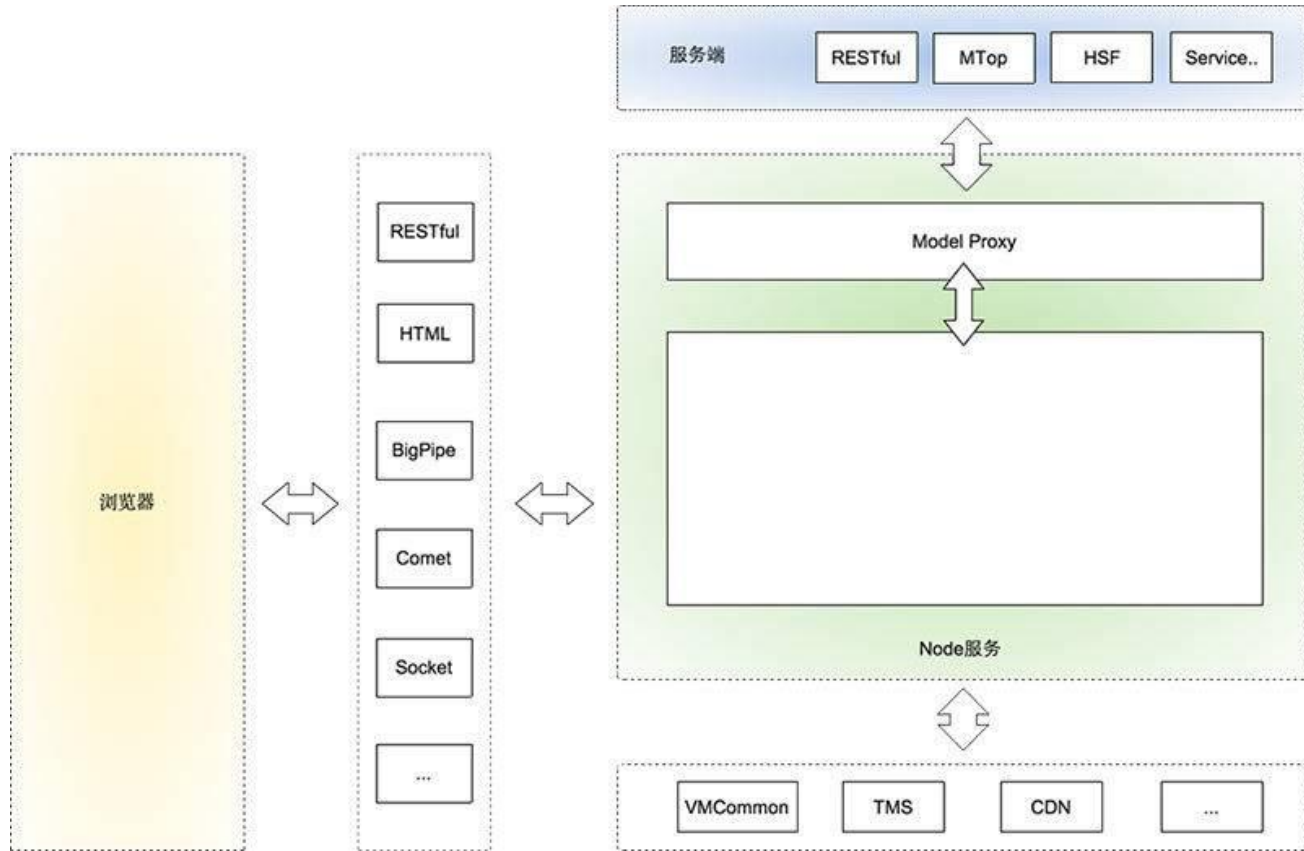
- SPA is an **individual** project!
- Front-end and back-end are two systems!

Node.Js



- Developing React and Vue also need a web server.
- NodeJs is a webserver powered by JavaScript and used for front-end developing
- Vue/React can be packaged to **static** files (mainly Js type) to run on any web server.

Revolution has **NO** ends

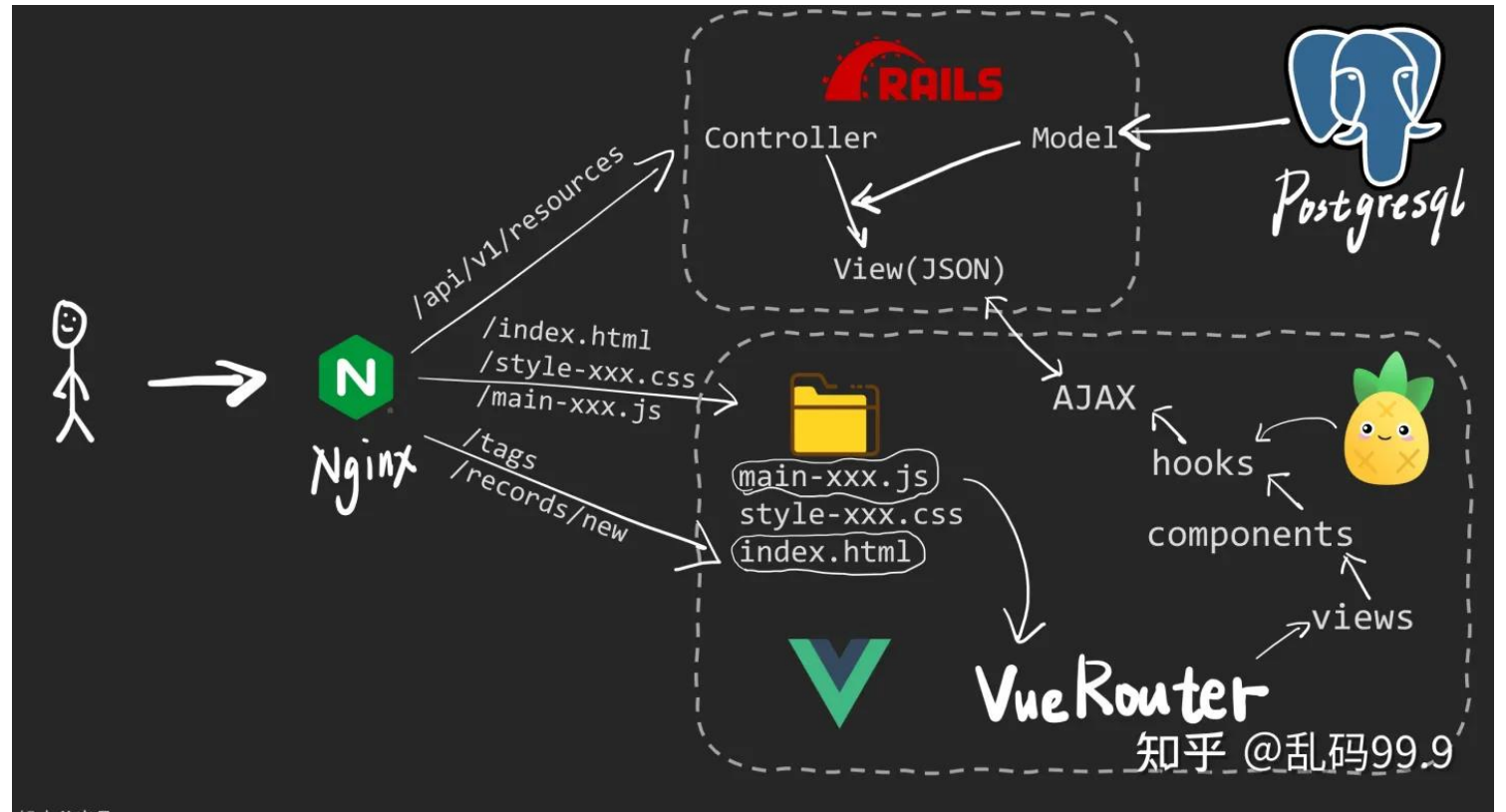


- NodeJS is also used as production server
- Insert NodeJS agent in the back-end
- Front-end extend to back-end things

<https://www.kancloud.cn/kancloud/midway/48192>

<https://www.it610.com/article/1280970709337653248.htm>

How to make the choice?



- Do **NOT** use **missiles** for a **mosquito**!